

MBA EM **ENGENHARIA**
DE SOFTWARE

PADRÕES DE PROJETOS (DESIGN PATTERNS) II

Prof^a. Dr^a. Rosana T. Vaccare Braga

MBAUSP
ESALQ

A responsabilidade pela idoneidade, originalidade e licitude dos conteúdos didáticos apresentados é do professor.

Proibida a reprodução, total ou parcial, sem autorização.

Lei nº 9610/98

Na aula anterior ...

- Reúso de software e seus benefícios
- Padrões e sua origem
- Elementos de um padrão, tipos de padrão
- Padrões de projeto: definições, vantagens, categorias, formatos de escrita
- Padrões GoF da categoria “de criação” e 2 padrões da categoria “estrutural”

		Propósito		
		De criação	Estrutural	Comportamental
Escopo	Classe	Método-fábrica	Adaptador	Interpretador Método gabarito
	Objeto	• Fábrica Abstrata • Construtor • Protótipo • Objeto Unitário (Singleton)	• Adaptador • Ponte • Composto • Decorador • Fachada • Peso-pena • Procurador	• Cadeia de Responsabilidade • Comando • Iterador • Mediador * • Memento * • Observador • Estado • Estratégia • Visitador *

Na aula de hoje

- Demais padrões da categoria “estrutural”;
- Padrões da categoria “comportamental”;
- Relacionamento entre os padrões de projeto;
- Conclusões
- Discussão: Impacto da Inteligência Artificial nos padrões de projeto



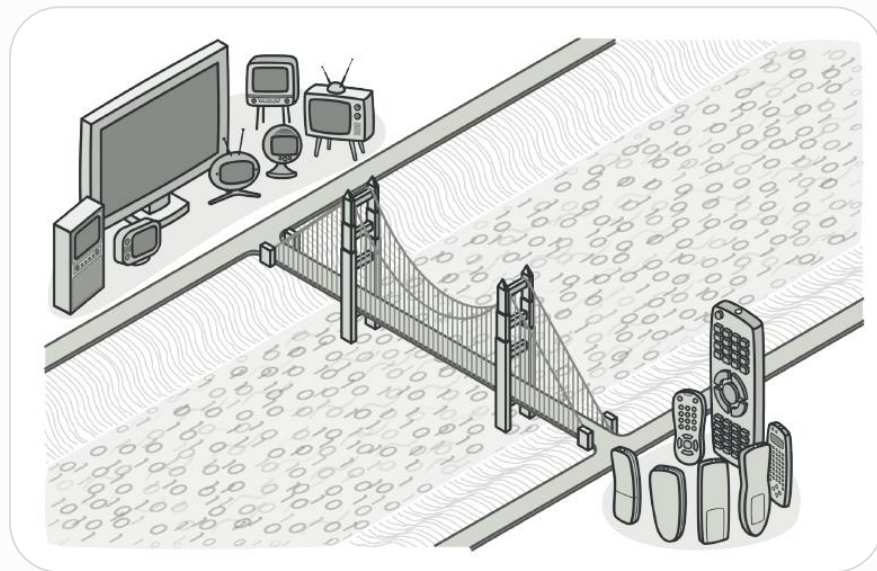
		Propósito		
		De criação	Estrutural	Comportamental
Escopo	Classe	Método-fábrica	Adaptador	Interpretador Método gabarito
	Objeto	- Fábrica Abstrata - Construtor - Protótipo - Objeto Unitário (Singleton)	- Adaptador - Ponte - Composto - Decorador - Fachada - Peso-pena - Procurador	- Cadeia de Responsabilidade - Comando - Iterador - Mediador - Memento - Observador - Estado - Estratégia - Visitador

PADRÕES da categoria “estrutural”

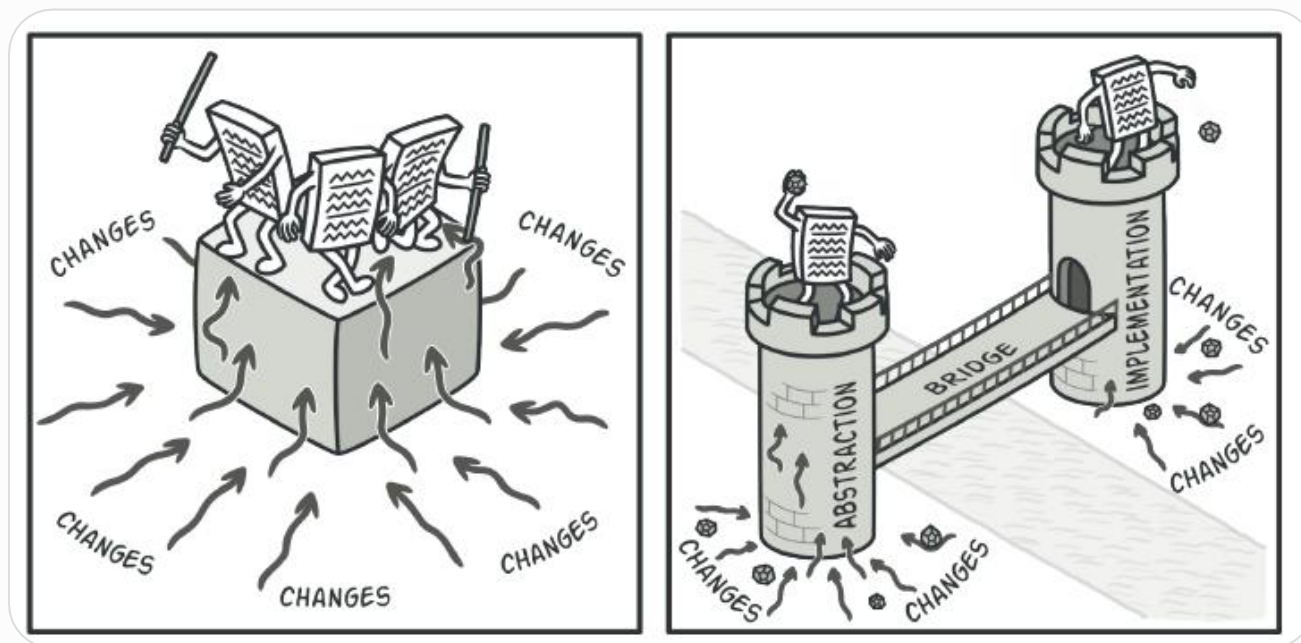
Preocupam-se com a maneira como ocorrem as composições de classes e objetos em estruturas maiores

Retomando da aula anterior: Padrão de Projeto Bridge

- **Classificação:** Padrão estrutural/Escopo de objetos
- **Intenção:** Desacoplar uma abstração de sua implementação para que as duas possam variar independentemente.

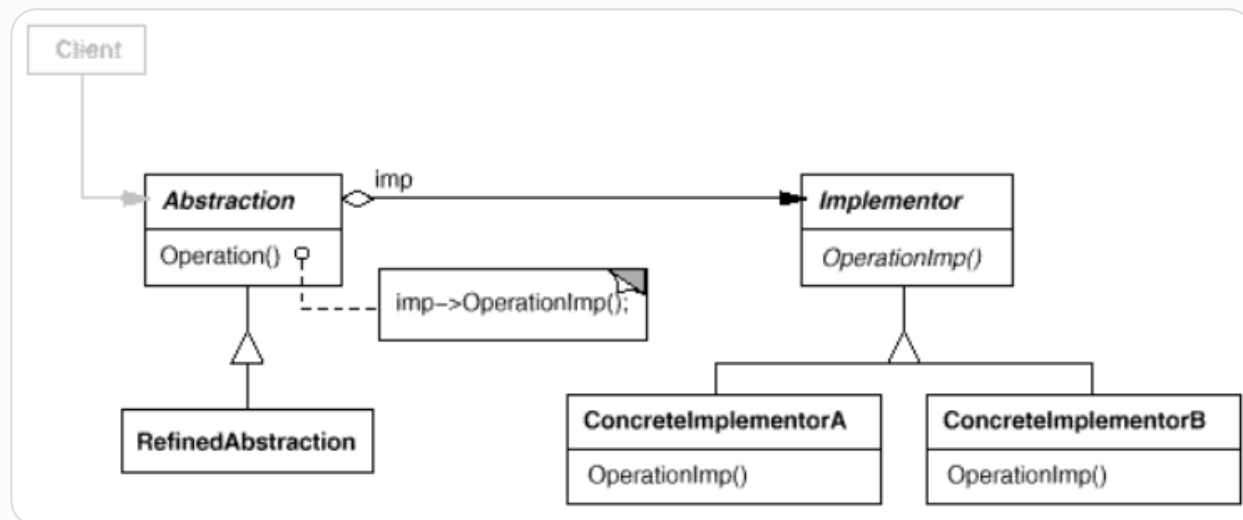


(Shvets, Alexander, 2018)

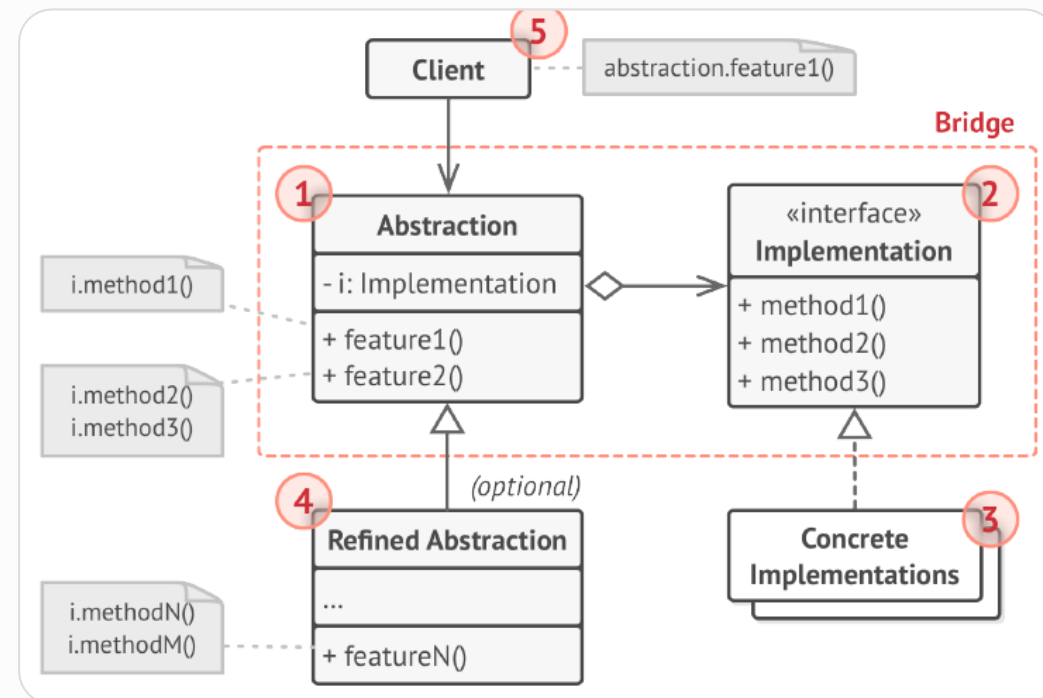


Padrão de Projeto Ponte

- Estrutura:



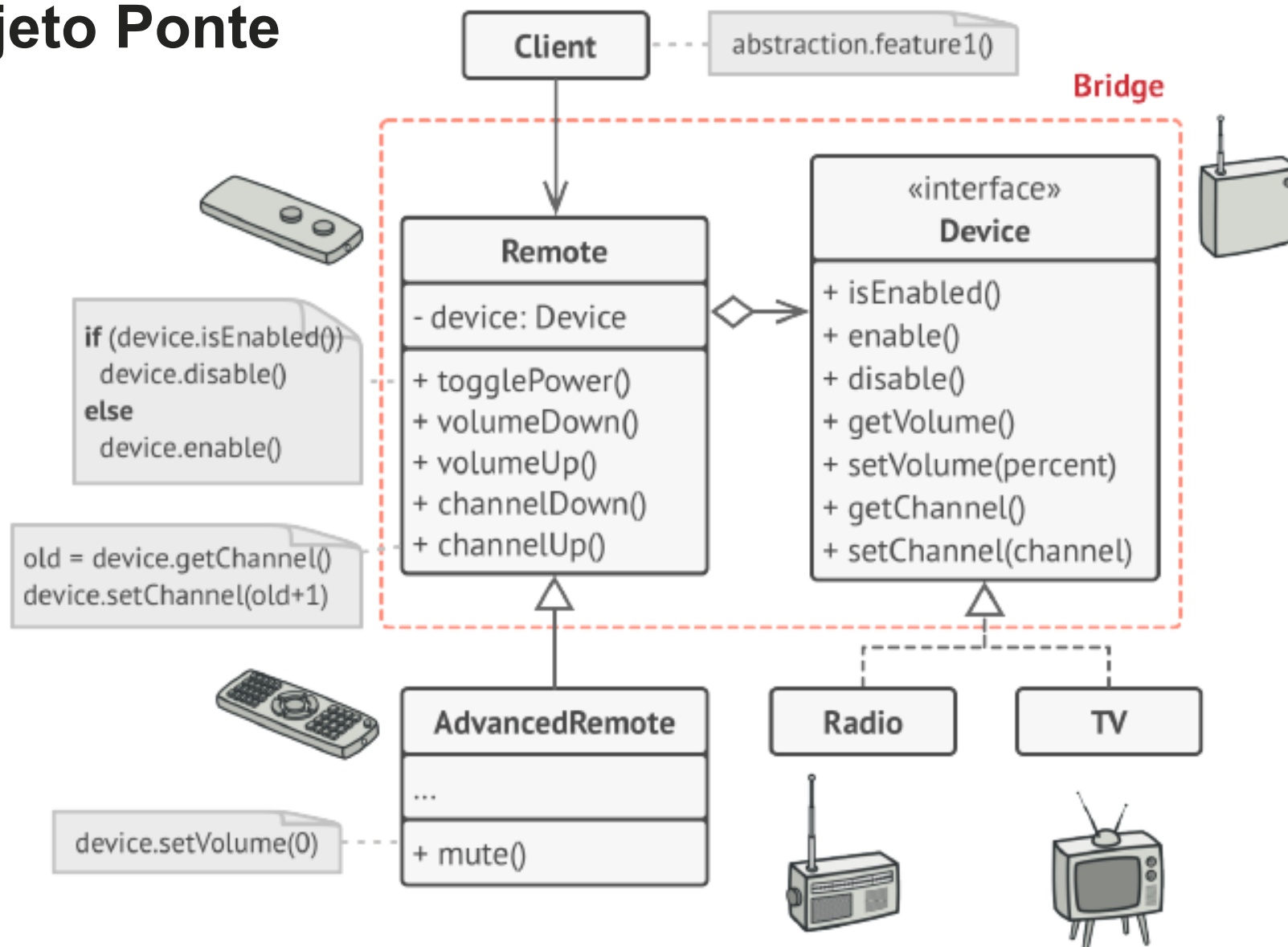
** Imagem capturada no livro versão html publicado em CD (Gamma et al 1995)



(Shvets, Alexander, 2018)

Padrão de Projeto Ponte

- Exemplo:



(Shvets, Alexander, 2018)

Padrão de Projeto Ponte

- Exemplo:

```
23
24 // You can extend classes from the abstraction hierarchy
25 // independently from device classes.
26 class AdvancedRemoteControl extends RemoteControl is
27     method mute() is
28         device.setVolume(0)
29
```

```
1 // The "abstraction" defines the interface for the "control"
2 // part of the two class hierarchies. It maintains a reference
3 // to an object of the "implementation" hierarchy and delegates
4 // all of the real work to this object.
5 class RemoteControl is
6     protected field device: Device
7     constructor RemoteControl(device: Device) is
8         this.device = device
9     method togglePower() is
10         if (device.isEnabled()) then
11             device.disable()
12         else
13             device.enable()
14     method volumeDown() is
15         device.setVolume(device.getVolume() - 10)
16     method volumeUp() is
17         device.setVolume(device.getVolume() + 10)
18     method channelDown() is
19         device.setChannel(device.getChannel() - 1)
20     method channelUp() is
21         device.setChannel(device.getChannel() + 1)
22
```

(Shvets, Alexander, 2018)

Padrão de Projeto Ponte

- Exemplo:

```
37 interface Device is
38     method isEnabled()
39     method enable()
40     method disable()
41     method getVolume()
42     method setVolume(percent)
43     method getChannel()
44     method setChannel(channel)
45
46
47 // All devices follow the same interface.
48 class Tv implements Device is
49     // ...
50
51 class Radio implements Device is
52     // ...
```

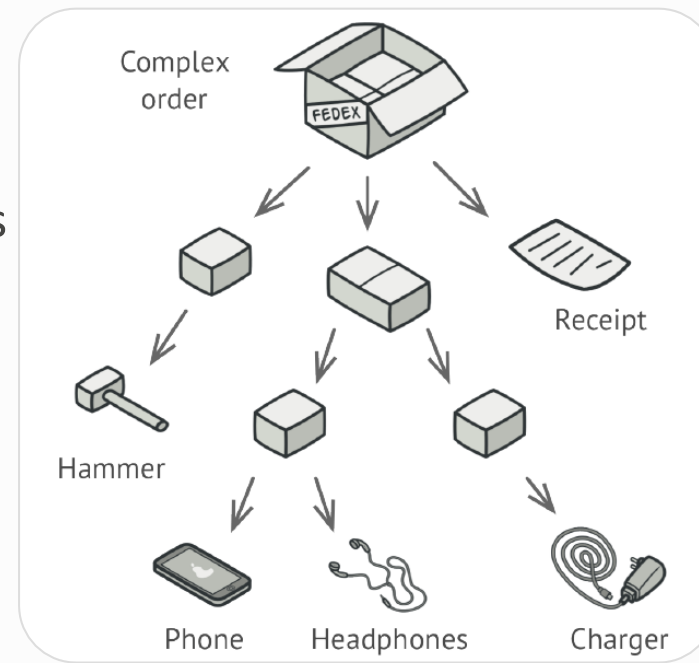
Padrão de Projeto Ponte

- Exemplo:

```
55  // Somewhere in client code.  
56  tv = new Tv()  
57  remote = new RemoteControl(tv)  
58  remote.togglePower()  
59  
60  radio = new Radio()  
61  remote = new AdvancedRemoteControl(radio)
```

Padrão de Projeto Composto (do inglês Composite)

- **Classificação:** Padrão de Criação/Escopo de objetos
- **Intenção:** Compor objetos em estruturas de árvore para representar hierarquias todo-parte. O Composto permite que o cliente trate objetos individuais e composições de objetos uniformemente.
- **Motivação:** Em aplicações gráficas, você pode construir diagramas complexos a partir de componentes simples. Pode agrupar componentes para criar outros componentes maiores, em vários níveis.
 - O problema é que o código que usa essas classes deve tratar de maneira diferente os objetos primitivos e os objetos que contêm outros objetos, o que torna o código mais complexo.

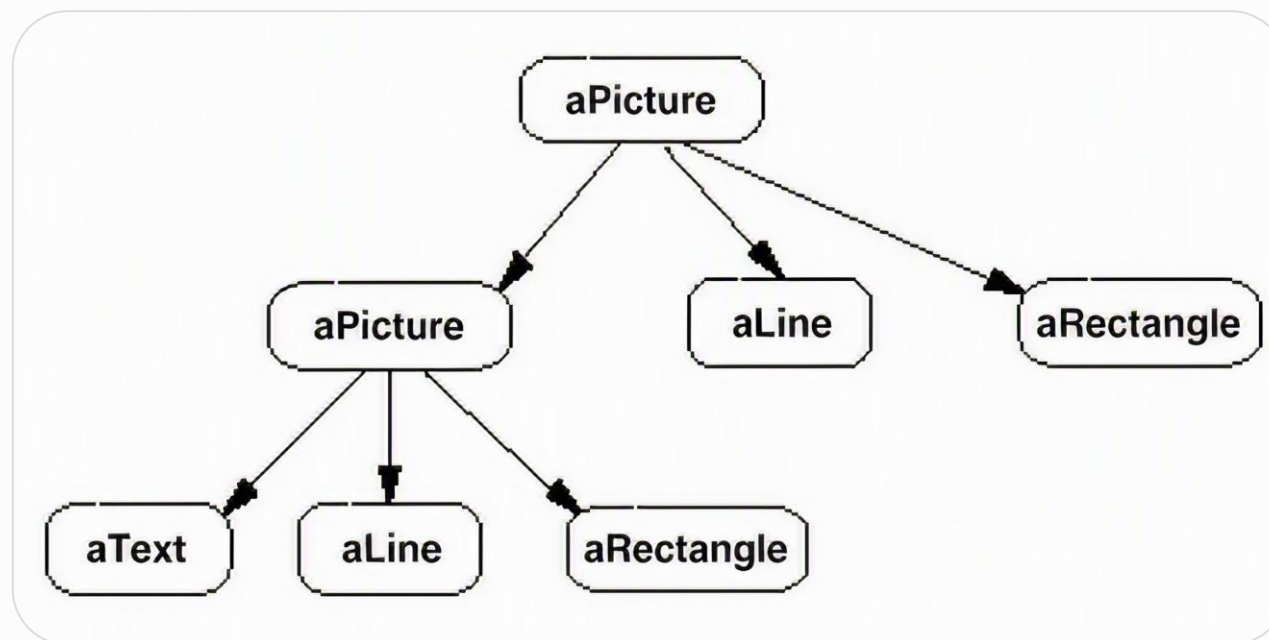
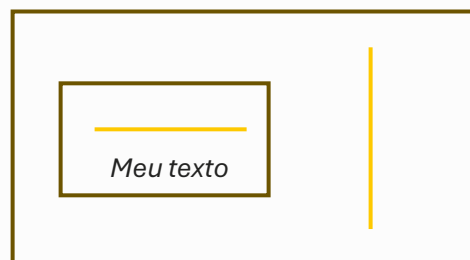


(Shvets, Alexander, 2018)

Enquete 6: Exemplos de Composto



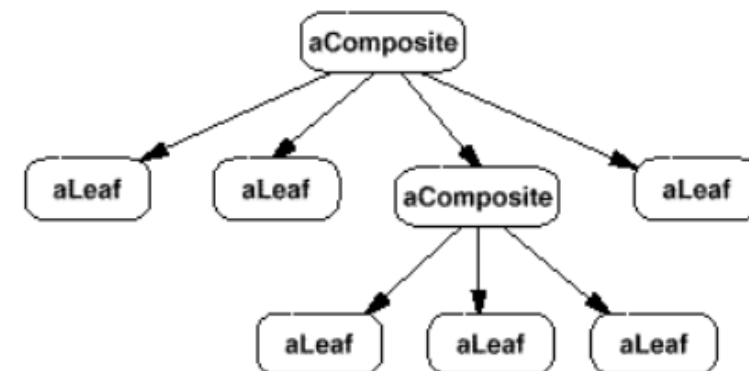
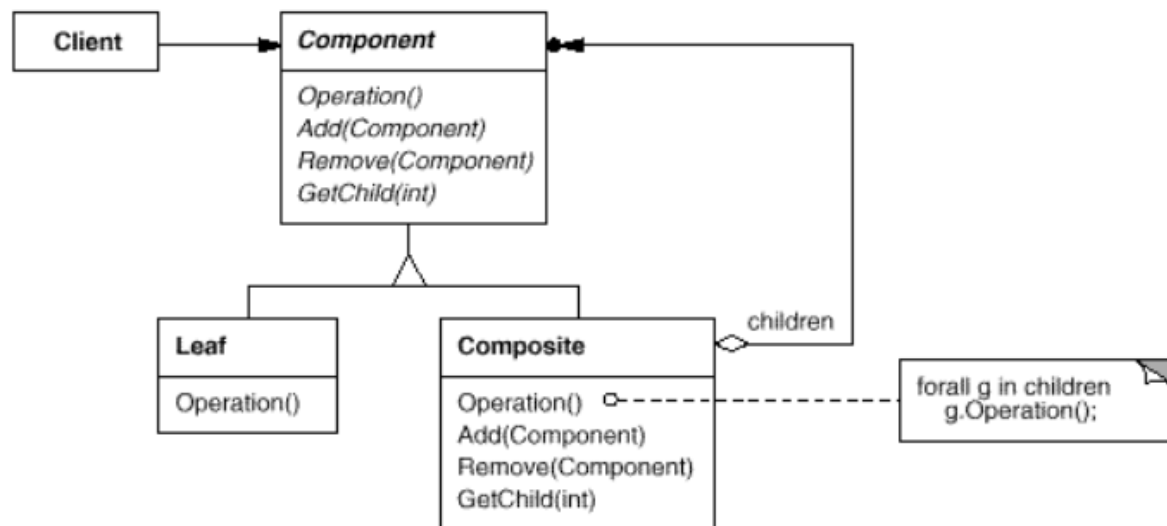
Outro exemplo do Padrão de Projeto Composto



** Imagem capturada no livro versão html publicado em CD (Gamma et al 1995)*

Padrão de Projeto Composto

- Aplicabilidade
- Use o padrão Composto quando:
 - você quer representar hierarquias todo-partes.
 - você quer que os clientes sejam capazes de ignorar a diferença entre composições de objetos e objetos individuais. Os clientes tratarão todos os objetos na estrutura composta uniformemente.
- Estrutura:



*** Imagens capturadas no livro versão html publicado em CD (Gamma et al 1995)*

Padrão de Projeto Composto

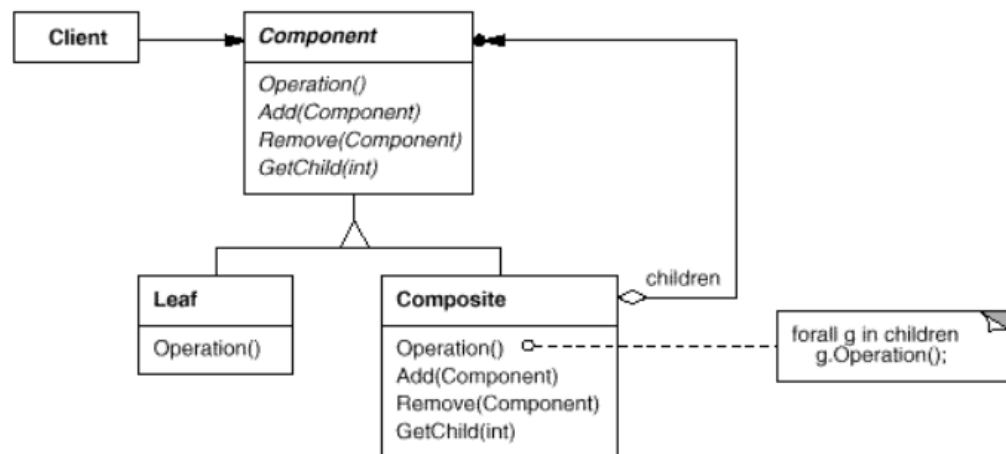
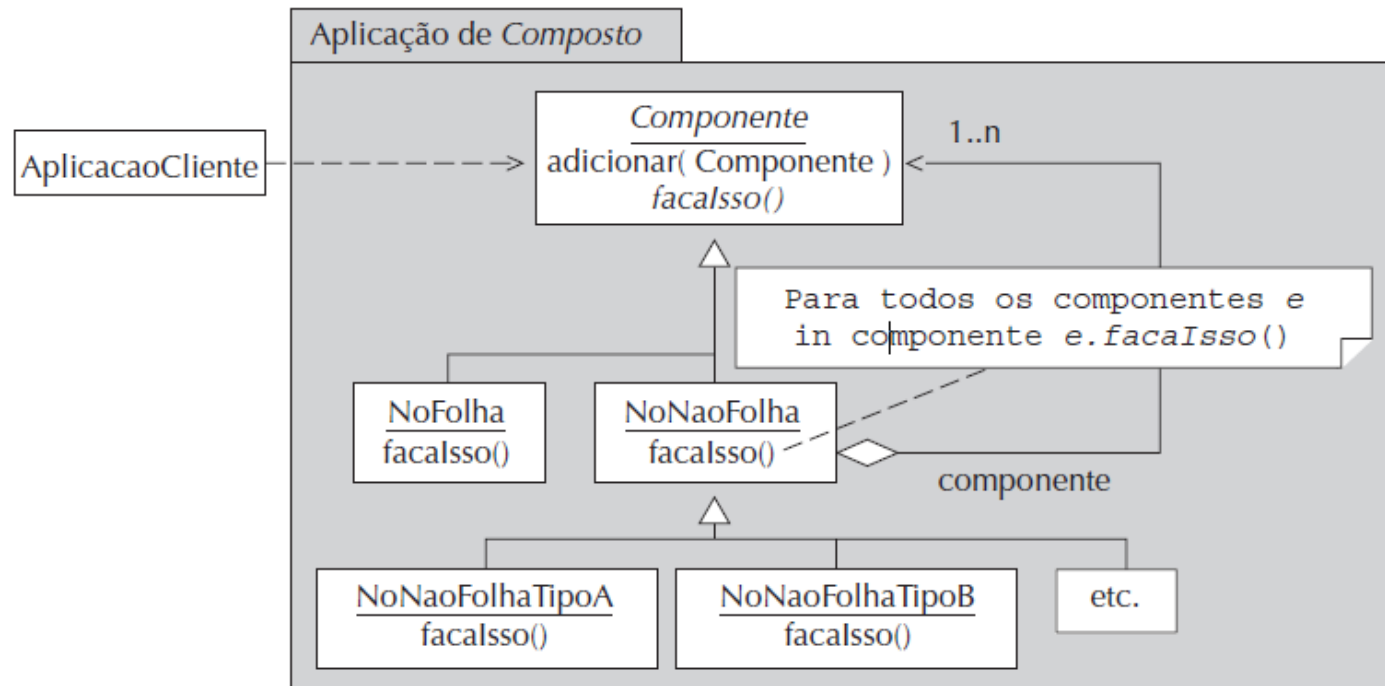


Imagem capturada no livro versão html publicado em CD (Gamma et al 1995)



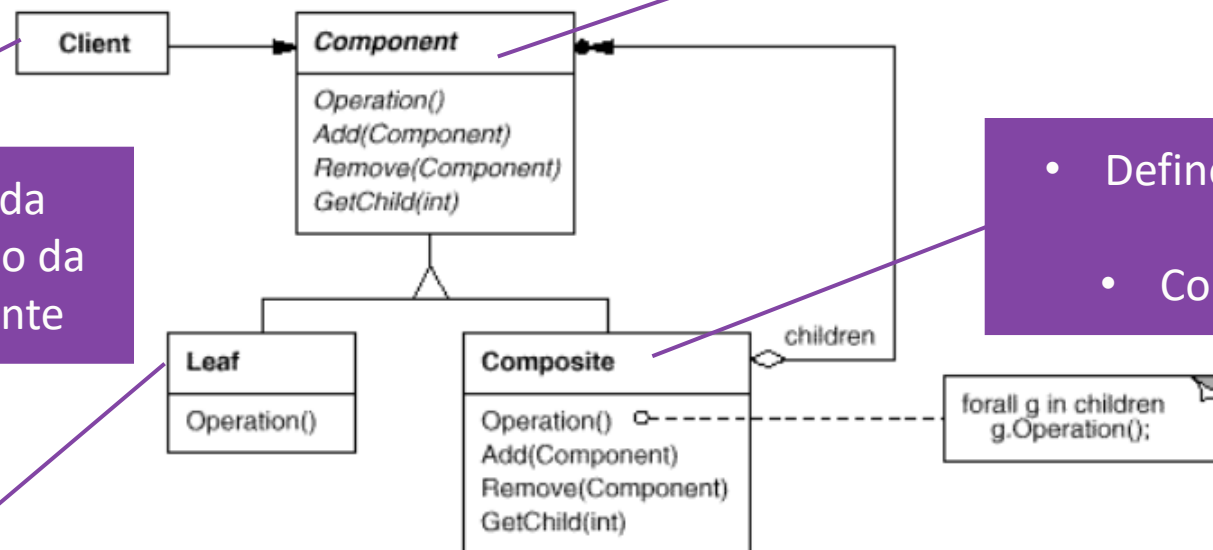
(Braude, Eric, 2005)

Padrão de Projeto Composto

- Participantes:

- Declara interface para objetos da composição e para gerenciar componentes-filho
- Comportamento default fica aqui, se for o caso

- Manipula objetos da composição por meio da interface Componente

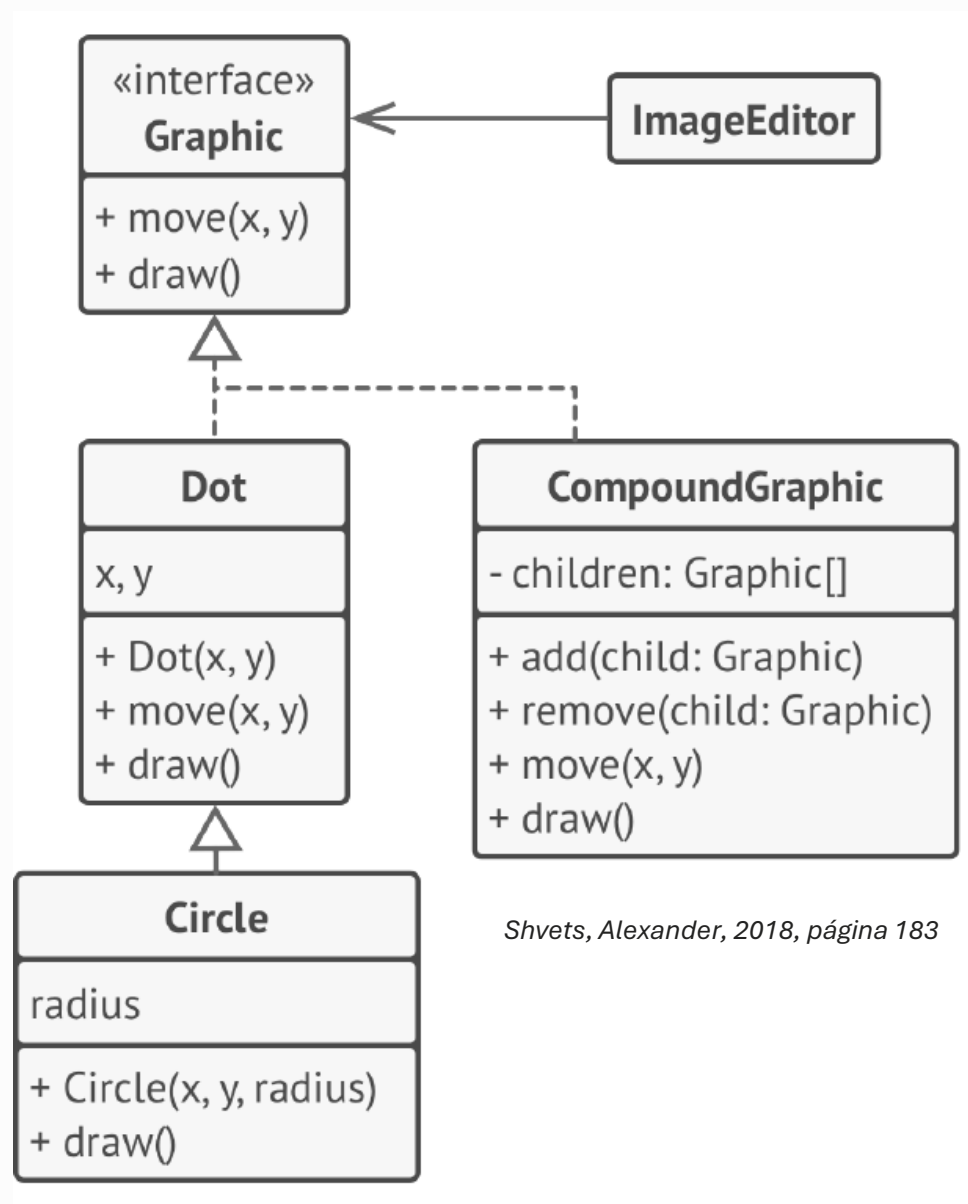


- Define comportamento para objetos que têm filhos
- Conhece os componentes filho

- Representa objetos folha (que não tem filhos)
- Define comportamento para objetos primitivos na composição

*** Imagem capturada no livro versão html publicado em CD (Gamma et al 1995)*

Exemplo de código do Padrão de Projeto Composto



Exemplo de código do Padrão de Projeto Composto

```
1  // The component interface declares common operations for both
2  // simple and complex objects of a composition.
3  interface Graphic is
4      method move(x, y)
5      method draw()
6
7  // The leaf class represents end objects of a composition. A
8  // leaf object can't have any sub-objects. Usually, it's leaf
9  // objects that do the actual work, while composite objects only
10 // delegate to their sub-components.
11 class Dot implements Graphic is
12     field x, y
13
14     constructor Dot(x, y) { ... }
15
16     method move(x, y) is
17         this.x += x, this.y += y
18
19     method draw() is
20         // Draw a dot at X and Y.
21
22 // All component classes can extend other components.
23 class Circle extends Dot is
24     field radius
25
26
```

Shvets, Alexander, 2018, página 184

Exemplo de código do Padrão de Projeto Composto

```
27     constructor Circle(x, y, radius) { ... }
28
29     method draw() is
30         // Draw a circle at X and Y with radius R.
31
32     // The composite class represents complex components that may
33     // have children. Composite objects usually delegate the actual
34     // work to their children and then "sum up" the result.
35     class CompoundGraphic implements Graphic is
36         field children: array of Graphic
37
38         // A composite object can add or remove other components
39         // (both simple or complex) to or from its child list.
40         method add(child: Graphic) is
41             // Add a child to the array of children.
42
43         method remove(child: Graphic) is
44             // Remove a child from the array of children.
45
46         method move(x, y) is
47             foreach (child in children) do
48                 child.move(x, y)
49
50         // A composite executes its primary logic in a particular
51         // way. It traverses recursively through all its children,
52         // collecting and summing up their results. Since the
53         // composite's children pass these calls to their own
54         // children and so forth, the whole object tree is traversed
55         // as a result.
56         method draw() is
57             // 1. For each child component:
58             //     - Draw the component.
```

Shvets, Alexander, 2018, página 185

Exemplo de código do Padrão de Projeto Composto

```
59      //      - Update the bounding rectangle.
60      // 2. Draw a dashed rectangle using the bounding
61      // coordinates.
62
63
64      // The client code works with all the components via their base
65      // interface. This way the client code can support simple leaf
66      // components as well as complex composites.
67      class ImageEditor is
68          method load() is
69              all = new CompoundGraphic()
70              all.add(new Dot(1, 2))
71              all.add(new Circle(5, 3, 10))
72              // ...
73
74      // Combine selected components into one complex composite
75      // component.
76      method groupSelected(components: array of Graphic) is
77          group = new CompoundGraphic()
78          group.add(components)
79          all.remove(components)
80          all.add(group)
81      // All components will be drawn.
82      all.draw()
```

Shvets, Alexander, 2018, página 186

Consequências de uso do Composto

- Objetos primitivos podem ser compostos em objetos mais complexos, que por sua vez podem ser compostos, e assim por diante recursivamente:
 - onde quer que o código do cliente espere um objeto primitivo, ele também pode receber um objeto composto.
- Torna o cliente simples.
 - Os clientes não precisam saber se estão lidando com uma folha ou um componente composto
 - evita ter que escrever funções no estilo “do case”
- Torna mais fácil adicionar novos tipos de componentes: os clientes não precisam ser alterados para novas classes Componente.
- Pontos negativos:
 - o projeto pode tornar-se muito genérico.
 - fica mais difícil restringir os componentes de um composto. Por exemplo, se for necessário que seu composto tenha apenas certos componentes. Você não pode confiar no sistema de tipos da linguagem de programação para impor essas restrições, mas terá que usar verificações em tempo de execução.

Padrões relacionados ao Composto

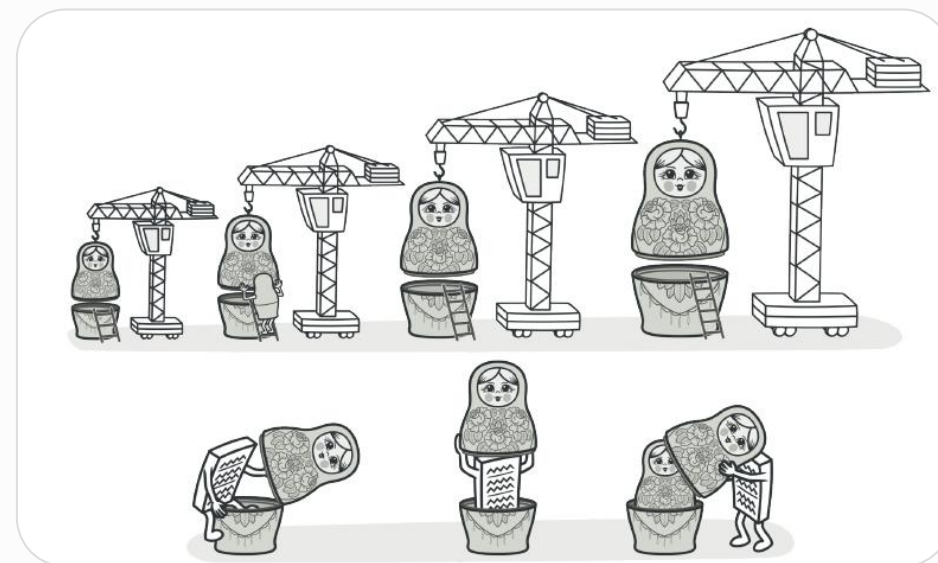
- Chain of Responsibility
- Decorator
- Flyweight
- Iterator
- Visitor

Retomando a enquete 6

- Você consegue desenhar um diagrama de classes aplicando o Padrão Composto em outro exemplo?

Padrão de Projeto Decorador (do inglês Decorator)

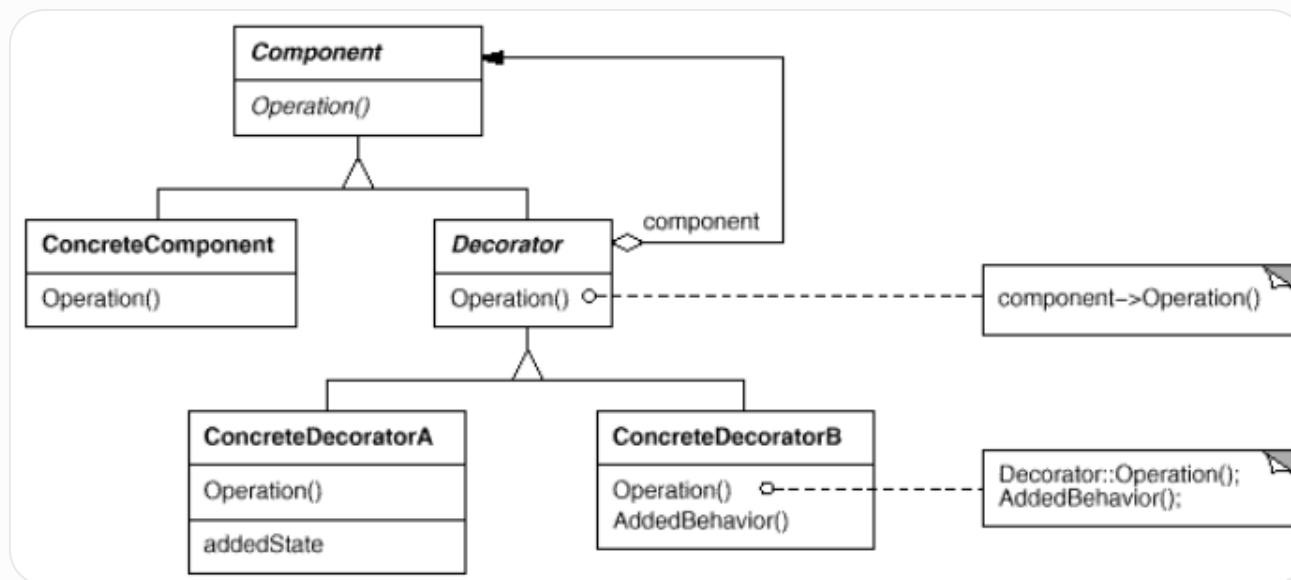
- **Classificação:** Padrão estrutural/Escopo de objetos
- **Intenção:** Atribuir responsabilidades adicionais a um objeto dinamicamente. Decoradores fornecem uma alternativa flexível à herança para estender a funcionalidade.
- **Aplicação:** Use Decorador para:
 - adicionar responsabilidades a objetos individuais de forma dinâmica e transparente, ou seja, sem afetar outros objetos.
 - Permitir que responsabilidades possam ser retiradas.
 - Evitar que um grande número de extensões independentes produza uma explosão de subclasses para dar suporte a cada combinação.



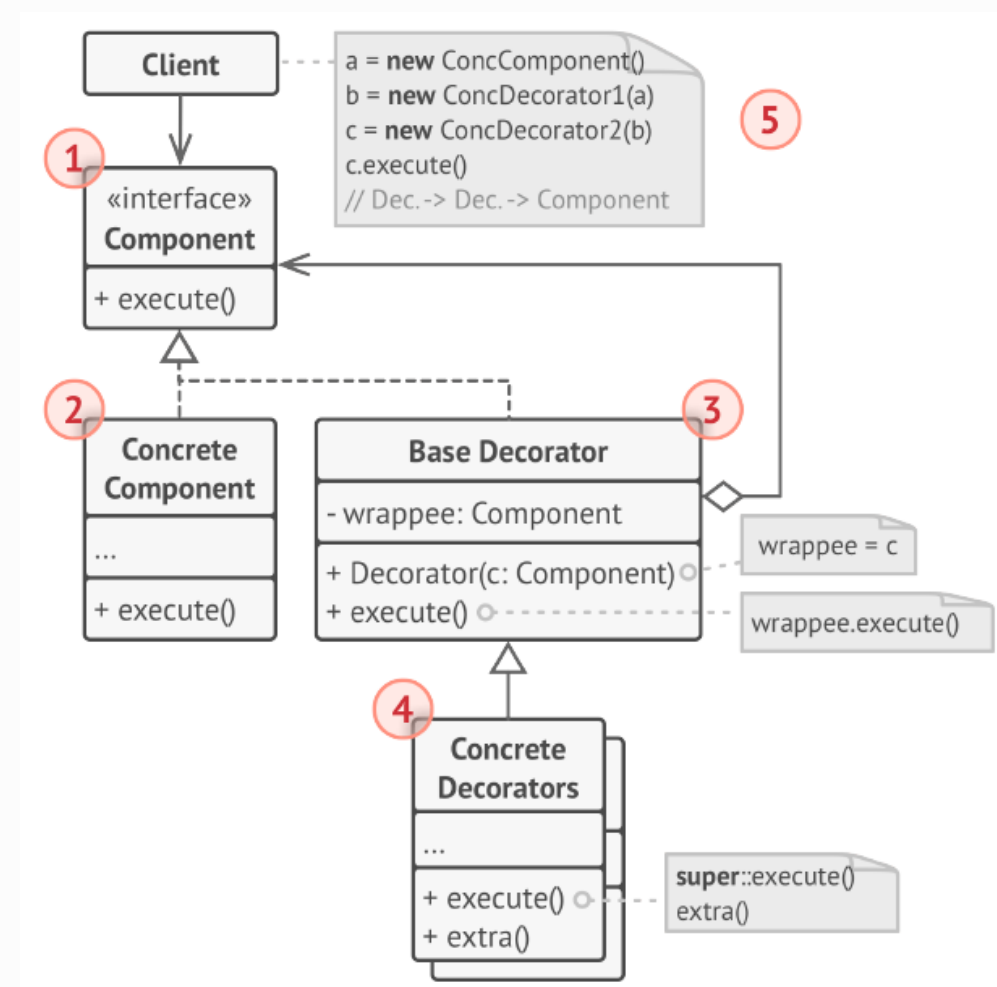
(Shvets, Alexander, 2018)

Padrão de Projeto Decorador

- Estrutura:

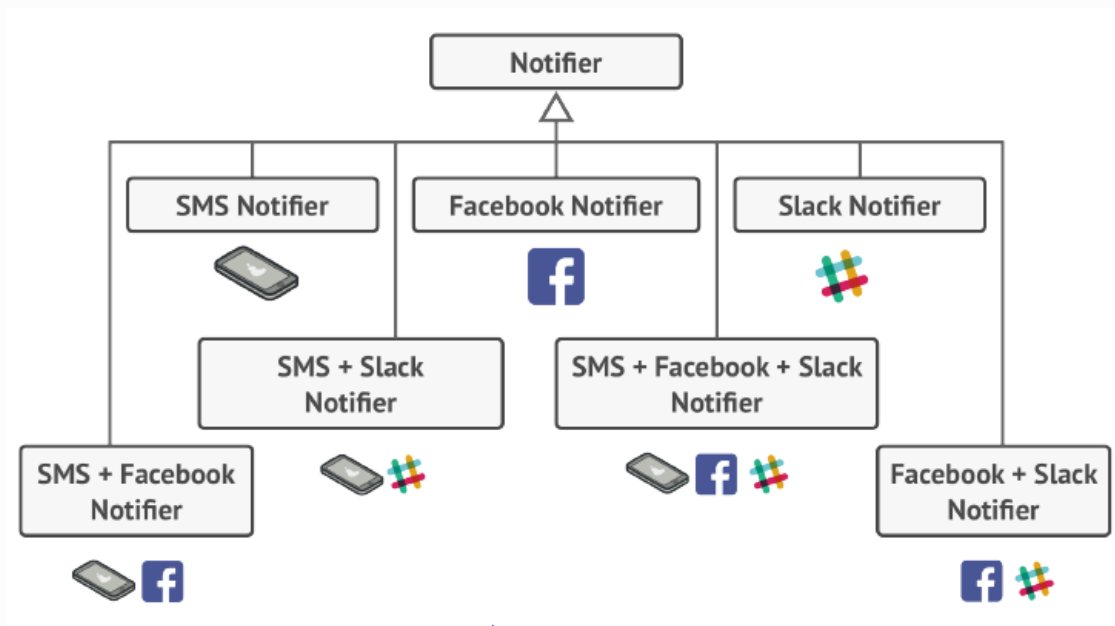


*** Imagem capturada no livro versão html
publicado em CD (Gamma et al 1995)*

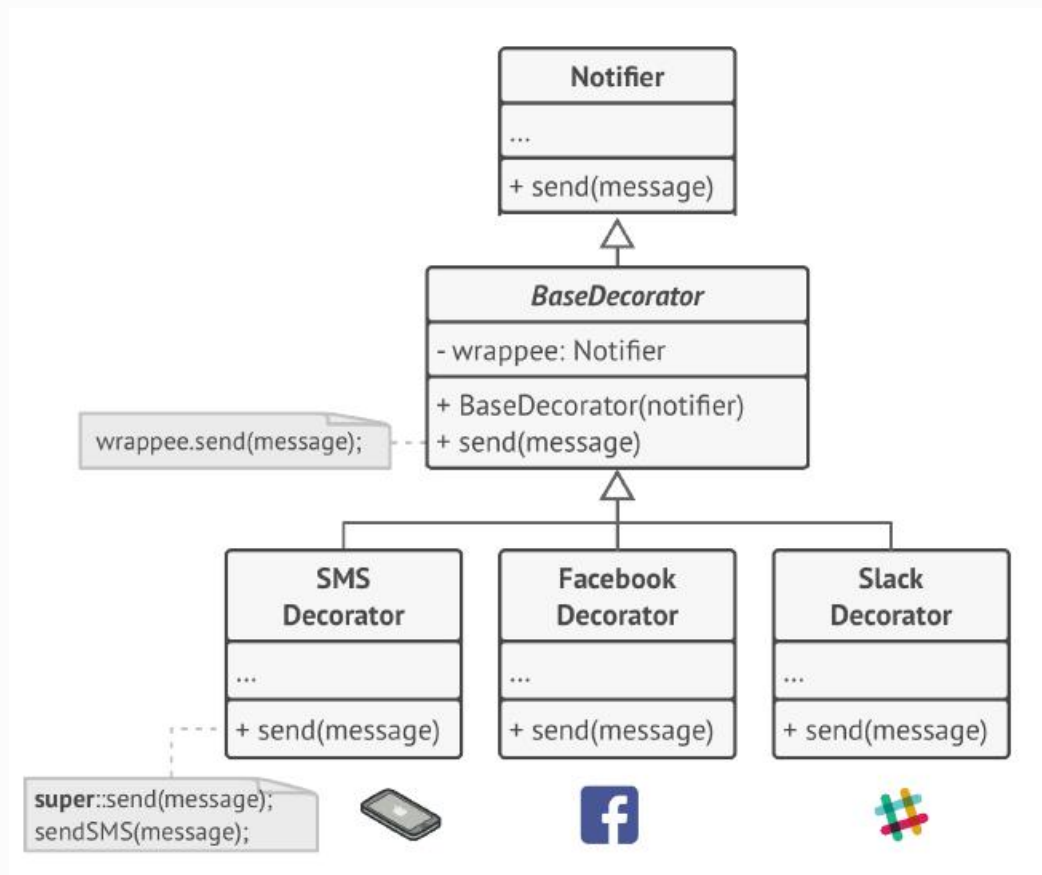


(Shvets, Alexander, 2018)

Padrão de Projeto Decorador



- Sem usar o Decorator

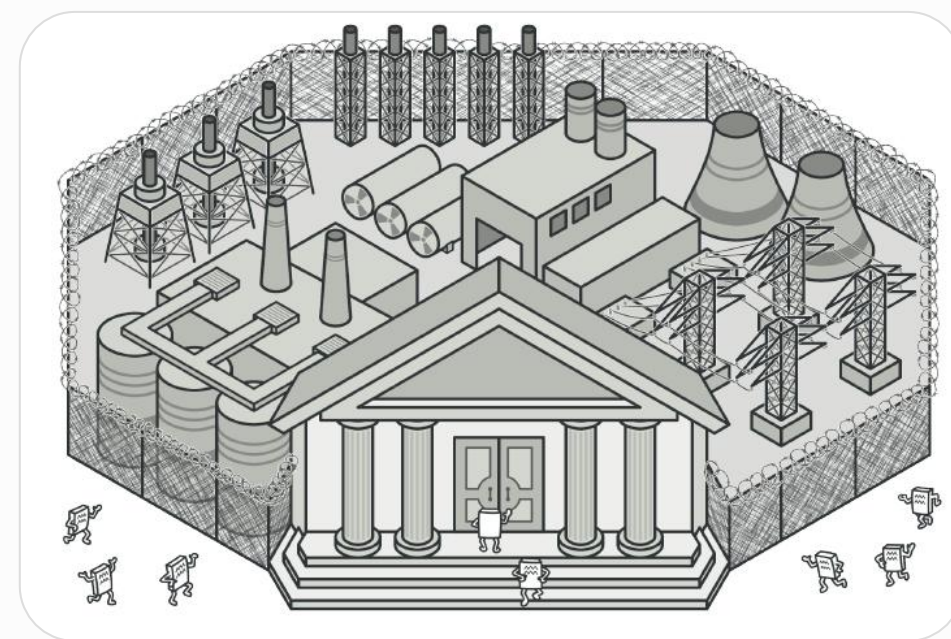


- Usando o Decorator

(Shvets, Alexander, 2018)

Padrão de Projeto Fachada (do inglês Facade)

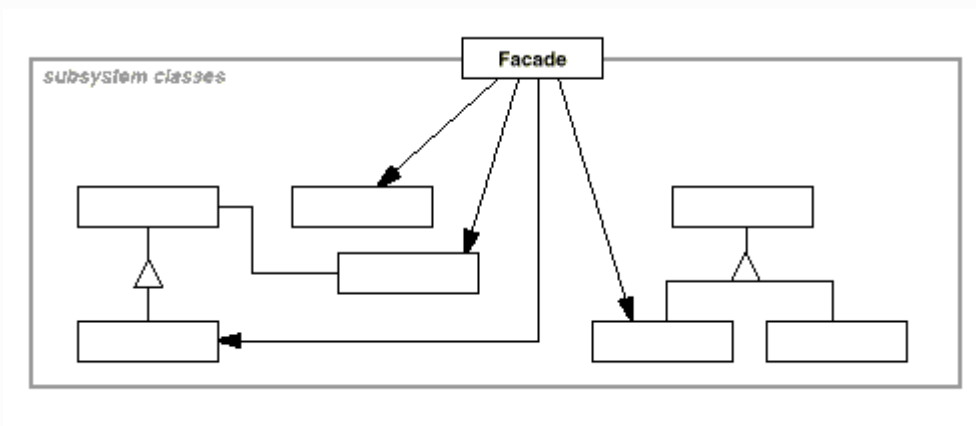
- **Classificação:** Padrão estrutural/Escopo de objetos
- **Intenção:** Fornecer uma interface unificada para um conjunto de interfaces em um subsistema. Fachadas definem uma interface de nível mais alto que torna o subsistema mais fácil de usar.
- **Aplicação:** você precisa integrar seu aplicativo ou sistema com uma biblioteca sofisticada que tem dezenas de recursos, mas você só precisa de uma parte da funcionalidade



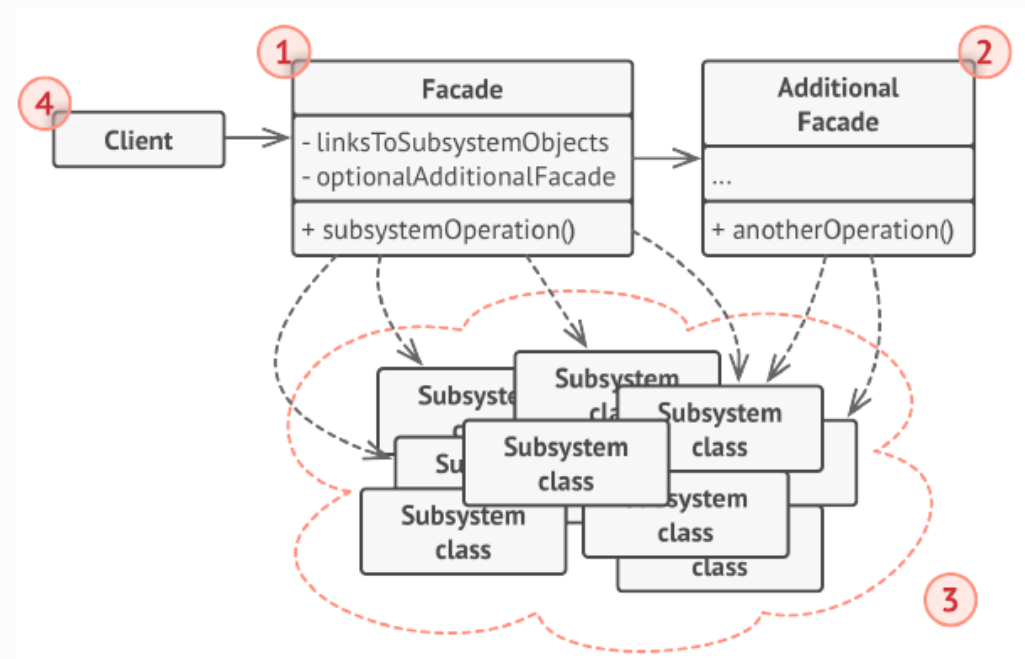
(Shvets, Alexander, 2018)

Padrão de Projeto Fachada

- Estrutura:



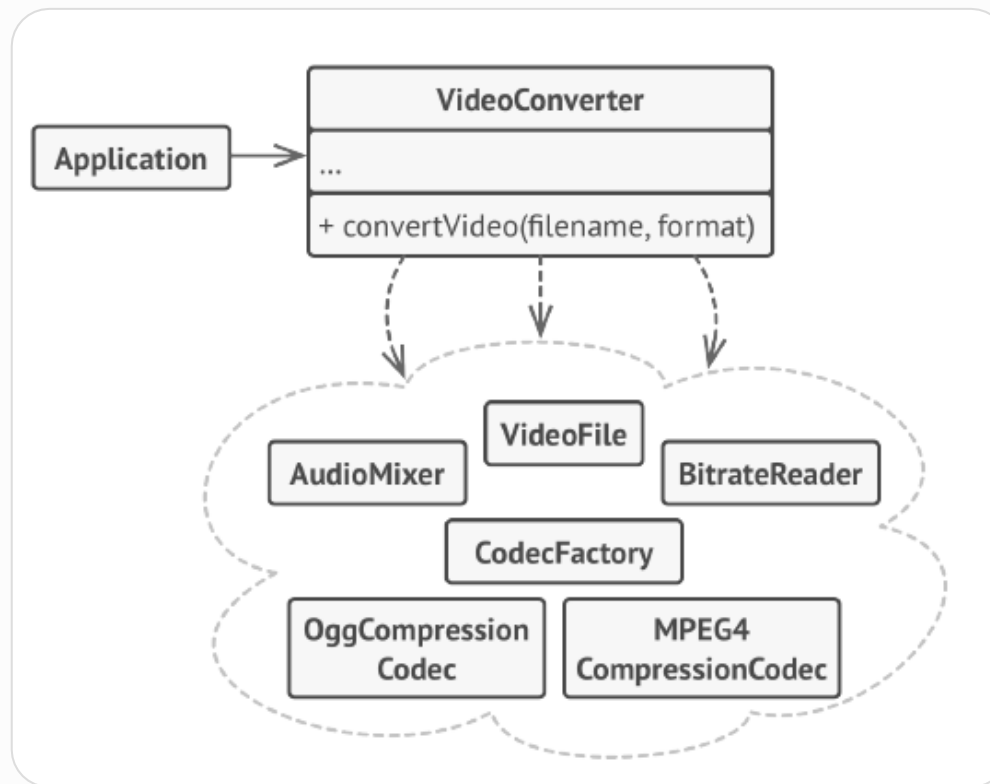
*** Imagem capturada no livro versão html publicado em CD (Gamma et al 1995)*



(Shvets, Alexander, 2018)

Padrão de Projeto Fachada

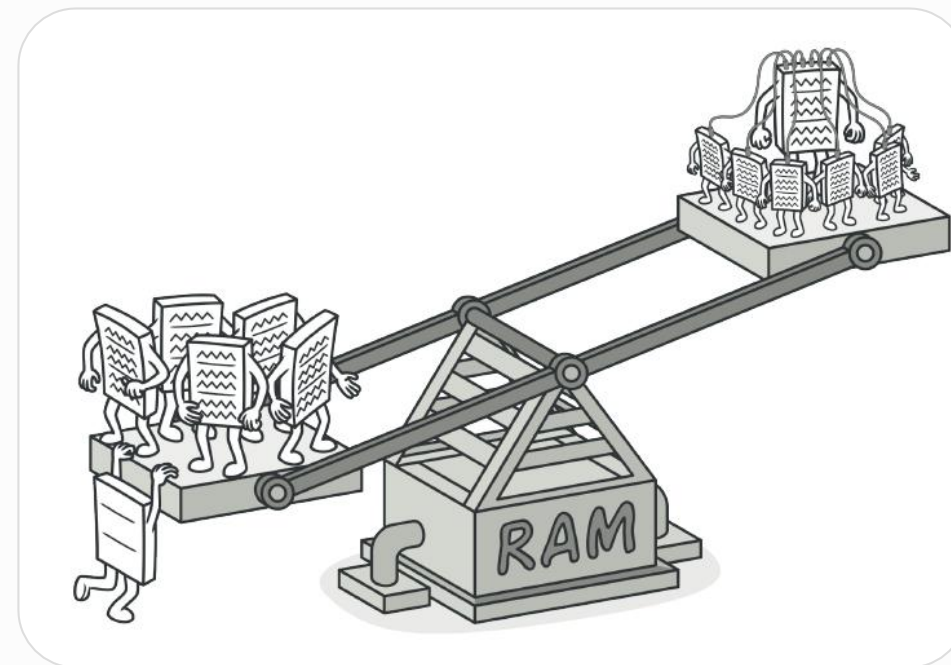
- Exemplo:



(Shvets, Alexander, 2018)

Padrão de Projeto Peso-pena (do inglês Flyweight)

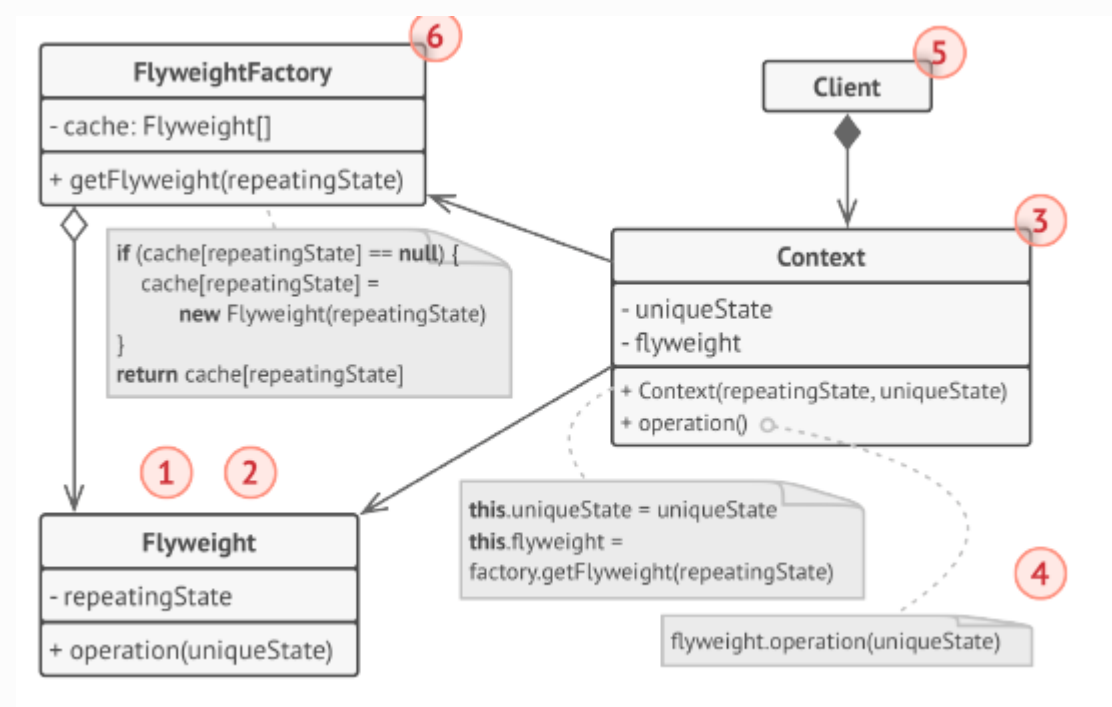
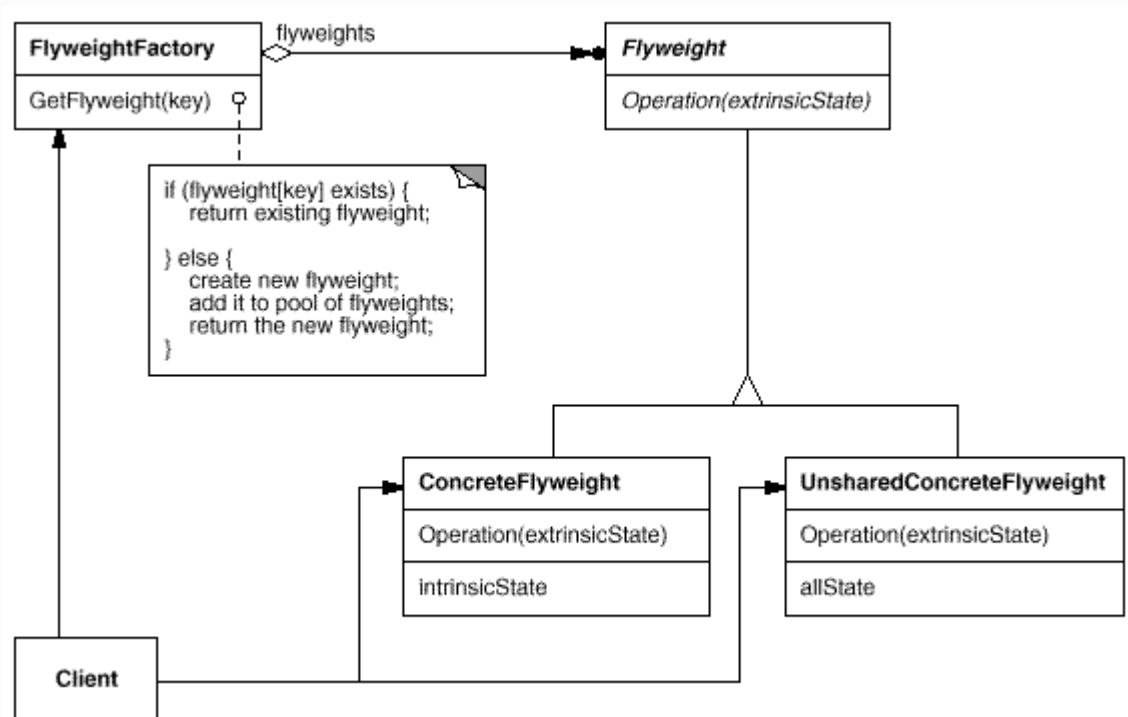
- **Classificação:** Padrão estrutural/Escopo de objetos
- **Intenção:** Usar o compartilhamento para oferecer suporte eficiente a um grande número de objetos de granularidade fina. Esse padrão permite, por exemplo, que você encaixe mais objetos na quantidade disponível de RAM compartilhando partes comuns de estado entre vários objetos em vez de manter todos os dados em cada objeto.
- **Aplicação:** Use o padrão Peso-pena apenas se seu programa precisar suportar um grande número de objetos similares que estão sobrecarregando a RAM disponível. Os objetos devem conter estados duplicados que podem ser extraídos e compartilhados entre vários objetos.



(Shvets, Alexander, 2018)

Padrão de Projeto Peso-pena

- Estrutura:

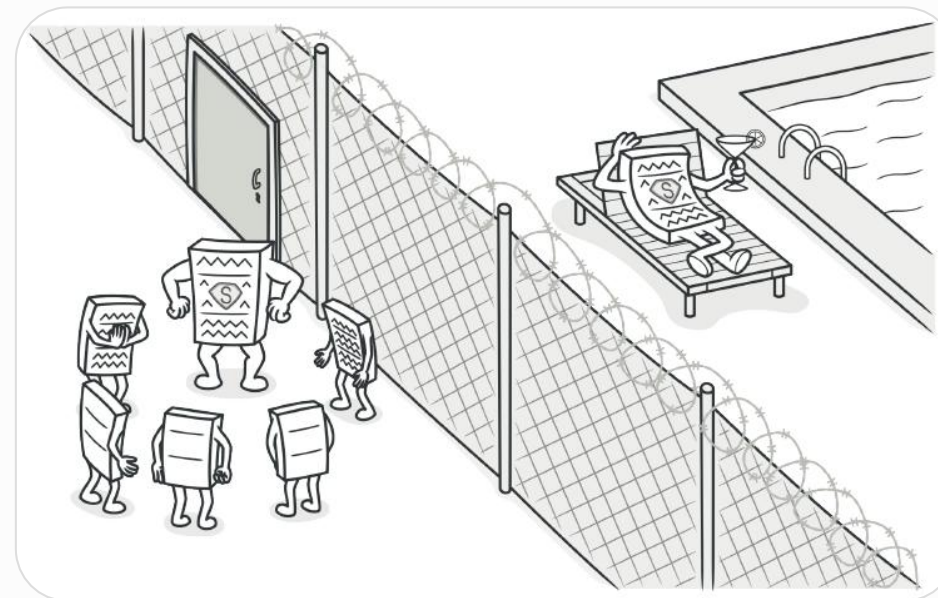


(Shvets, Alexander, 2018)

*** Imagem capturada no livro versão html
publicado em CD (Gamma et al 1995)*

Padrão de Projeto Procurador (do inglês Proxy)

- **Classificação:** Padrão estrutural/Escopo de objetos
- **Intenção:** Fornecer um substituto ou espaço reservado para outro objeto para controlar o acesso a ele.
- **Motivação:** Podemos querer controlar o acesso a um objeto, por exemplo, para adiar o custo total de sua criação e inicialização até que realmente precisemos usá-lo. O custo de criação do objeto em geral deve-se ao fato de que muitos atributos precisam ser inicializados, além de dependências com outros objetos que compõem o objeto.



(Shvets, Alexander, 2018)

Padrão de Projeto Proxy

- **Aplicabilidade**
- Usos do padrão Proxy:
 - *Remote proxy*: é necessário fazer referência a um objeto em um espaço de endereçamento diferente,
 - *Virtual proxy*: deseja-se criar objetos sob demanda (adiar a materialização do objeto até que seja realmente necessária), ou “*lazy initialization*”
 - *Protection proxy*: deseja-se manter um controle maior sobre o objeto original (controle de acesso por exemplo),
 - *Smart reference*: deseja-se manter uma referência “esperta” ao objeto (o ponteiro para o objeto realiza algum comportamento adicional sempre que o objeto é acessado).

Enquete 7: Utilização do Proxy



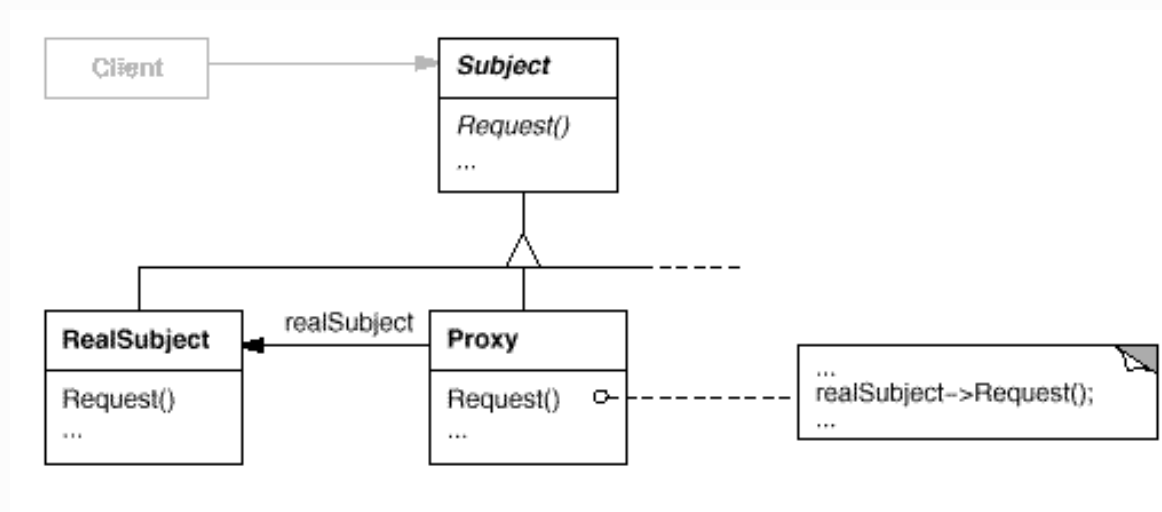
Padrão de Projeto Proxy

- **Solução**

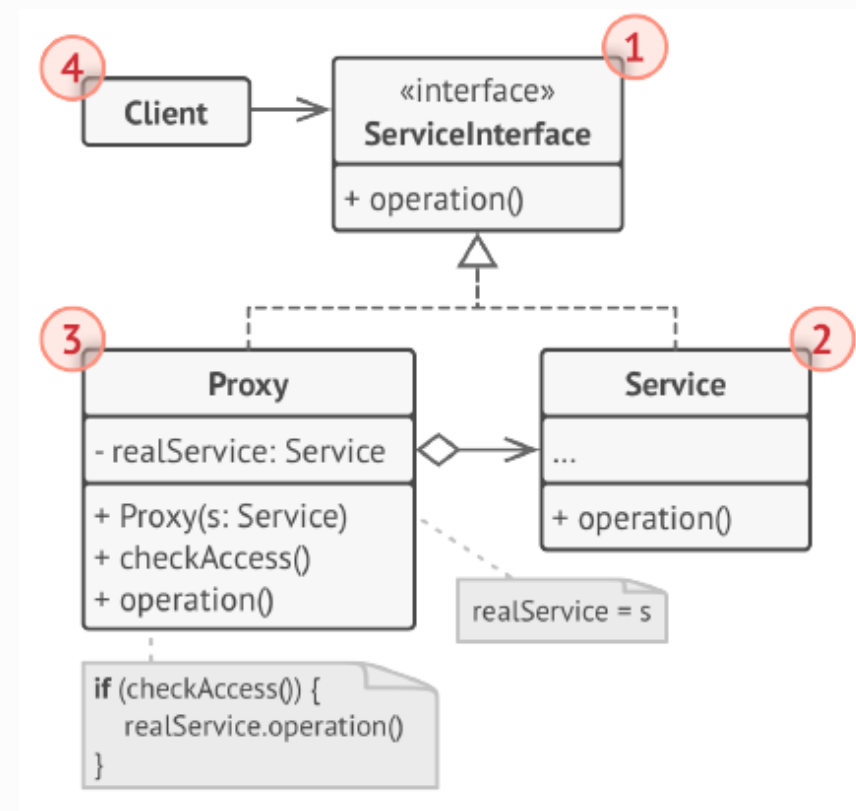
- Forneça um substituto, ou procurador, ou ainda um placeholder para o objeto, com a finalidade específica de controlar o acesso ao objeto.
- O Procurador mantém referência ao Sujeito Real e possui a mesma interface para que possa ser substituído pelo Sujeito Real.
- Em geral é o Procurador quem cria o sujeito Real, quando necessário.
- O Procurador delega solicitações ao Sujeito Real somente quando necessário. Isso depende do tipo de Procurador, sendo que vários tipos estão descritos na versão original do padrão Proxy (Gamma et al. 1995).

Padrão de Projeto Proxy

- Estrutura:



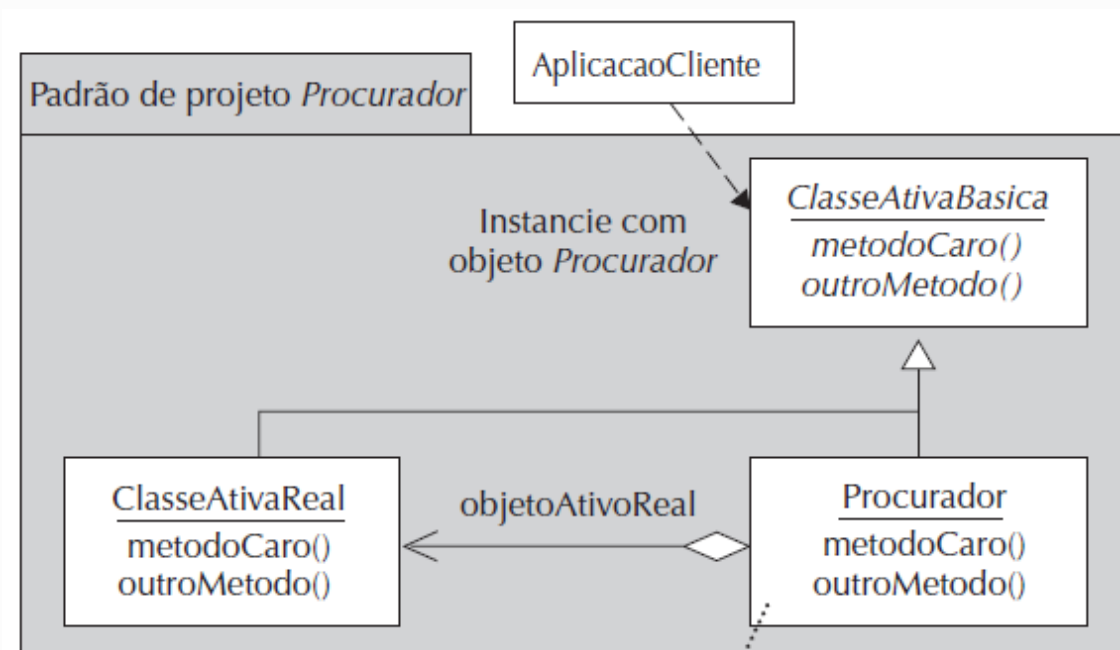
*** Imagem capturada no livro versão html
publicado em CD (Gamma et al 1995)*



(Shvets, Alexander, 2018)

Padrão de Projeto Proxy

- Estrutura:



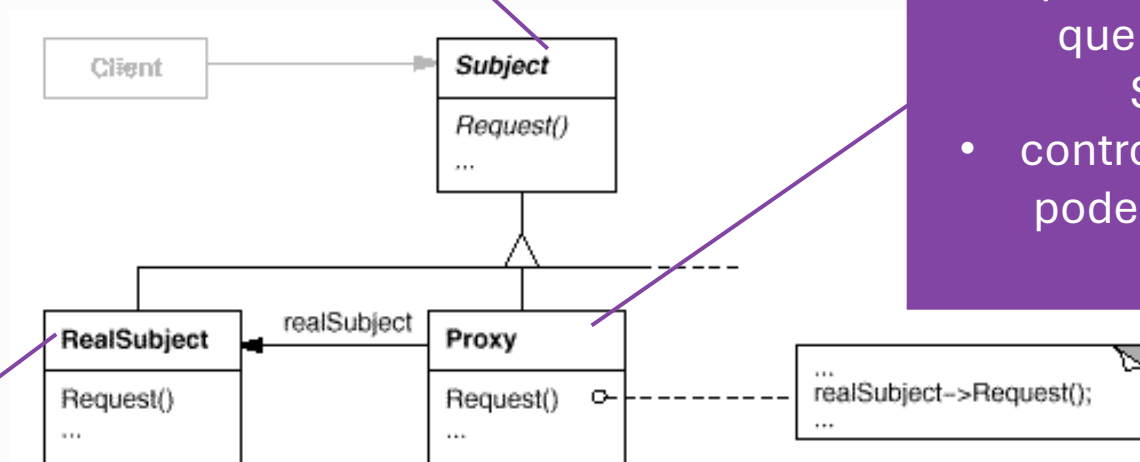
(Braude, Erick, 2005)

```
...//Uma maneira de verificar se realmente é necessário:
if(objetoAtivoReal==null)    //nunca referenciado
{
    objetoAtivoReal = obterObjetoAtivoReal();
    objetoAtivoReal.metodoCaro();
}
else//tenta evitar chamar o metodoCaro() real
```

Padrão de Projeto Proxy

- **Participantes:**

Define a interface comum para RealSubject e Proxy, de forma que o Proxy possa ser usado no lugar do RealSubject



- mantém uma referência que permite que o Proxy acesse RealSubject, já que as interfaces RealSubject e Subject são as mesmas.
- controla o acesso ao RealSubject e pode ser responsável por criá-lo e excluí-lo.

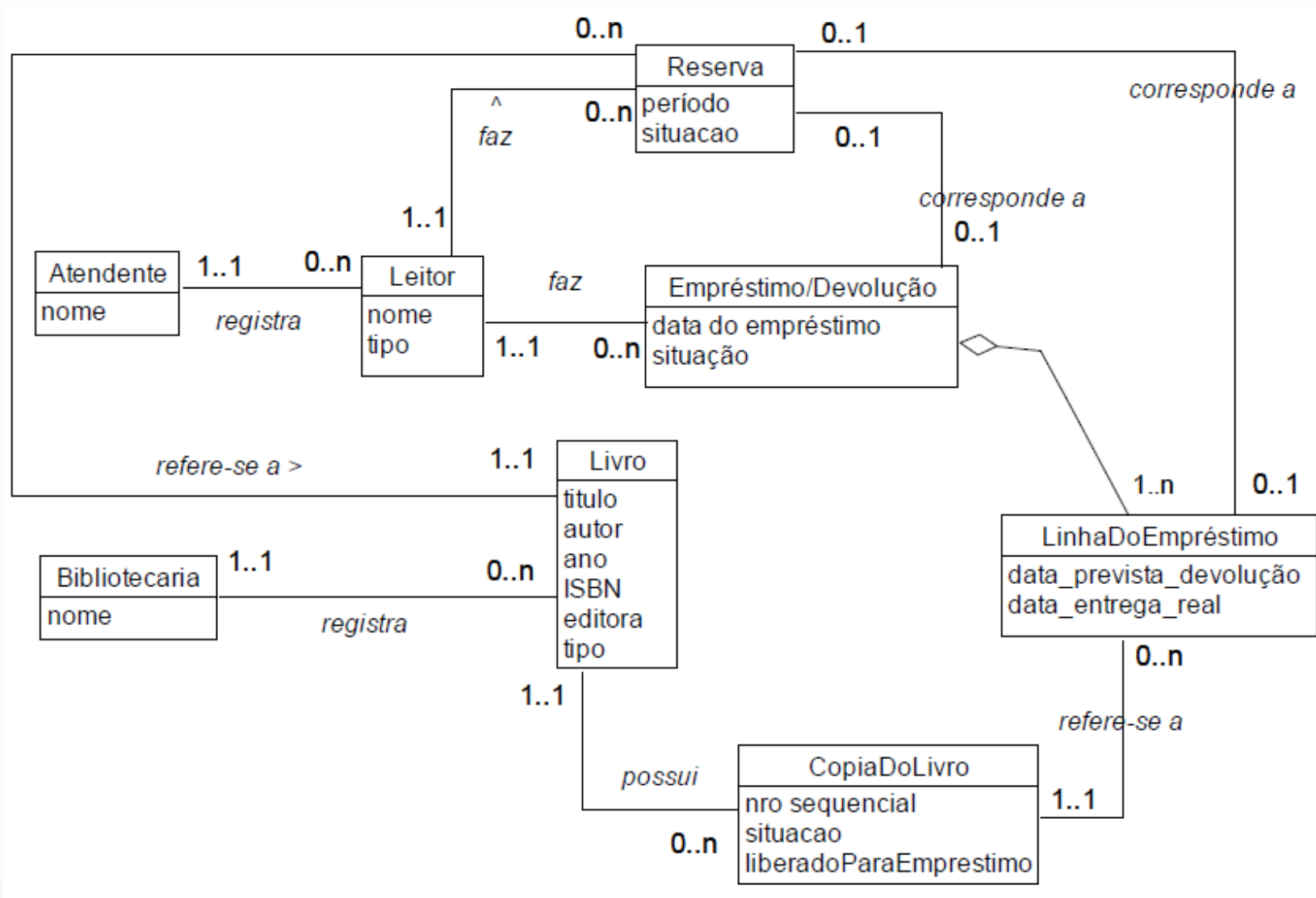
Define o RealSubject que o proxy representa

*** Imagem capturada no livro versão html publicado em CD (Gamma et al 1995)*

Padrão de Projeto Proxy

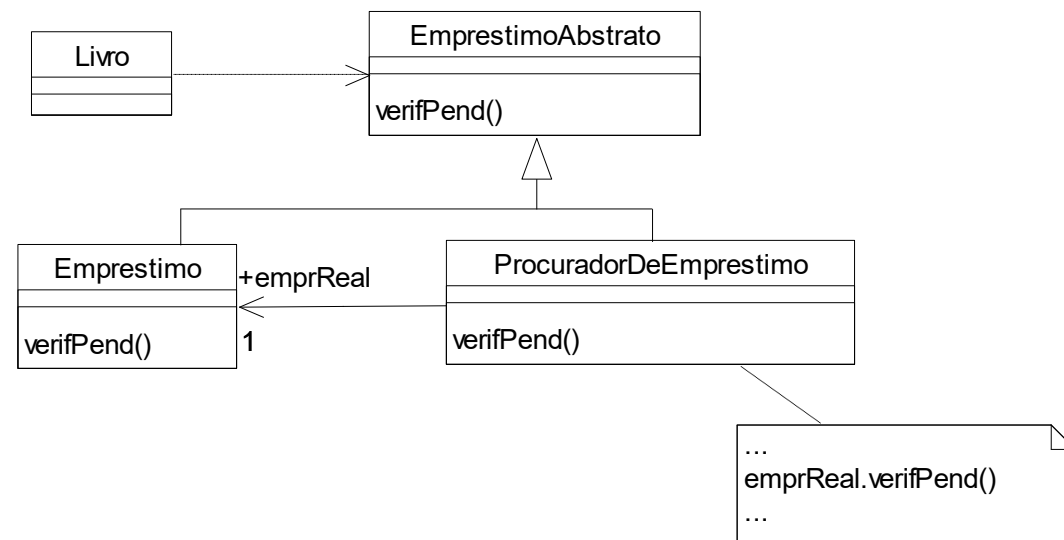
- **Exemplo: materialização e desmaterialização de objetos**
 - Materialização: trazer um registro de uma tabela na forma de um objeto
 - Desmaterialização: persistir um objeto como um registro em uma tabela
- Em um sistema de biblioteca, ao materializar um leitor, podemos no mesmo momento materializar os objetos referentes a todos os empréstimos daquele leitor.
 - No entanto, os objetos empréstimo talvez sejam materializados inutilmente, pois o sistema provavelmente não utilizará informações sobre esses objetos, desperdiçando tempo e espaço.

Padrão de Projeto Proxy



Padrão de Projeto Proxy

- A variante **Procurador Virtual** pode ser usada para solucionar este problema
- Quais objetos associados ao objeto X devem ser recuperados toda vez que um objeto X é recuperado?
- Para os objetos que não precisarão ser recuperados, cria-se um procurador.
 - Exemplo: ao materializar um Livro, deve-se também materializar seus empréstimos associados?
 - Resposta: não



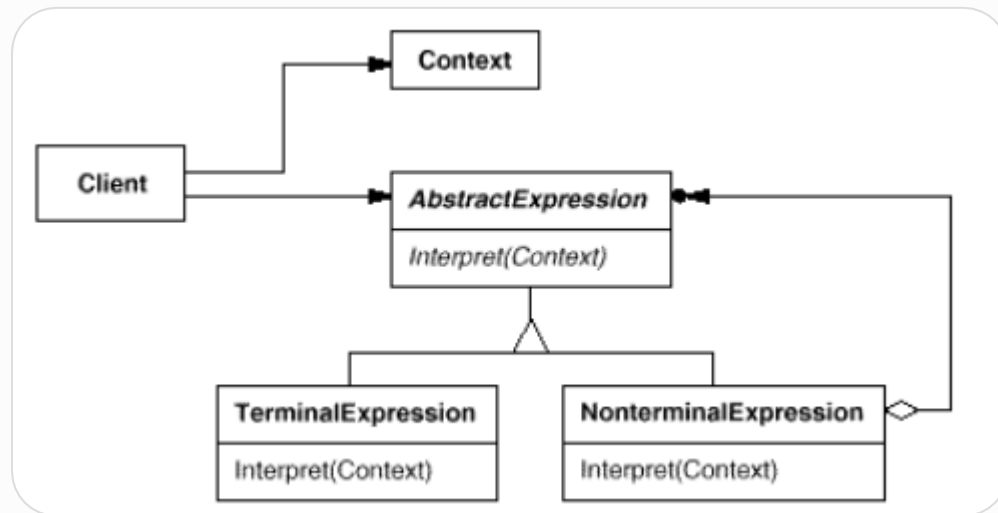
		Propósito		
		De criação	Estrutural	Comportamental
Escopo	Classe	Método-fábrica	Adaptador	Interpretador Método gabarito
	Objeto	- Fábrica Abstrata - Construtor - Protótipo - Objeto Unitário (Singleton)	- Adaptador - Ponte - Composto - Decorador - Fachada - Peso-pena - Procurador	- Cadeia de Responsabilidade - Comando - Iterador - Mediador - Memento - Observador - Estado - Estratégia - Visitador

PADRÕES da categoria “comportamental”

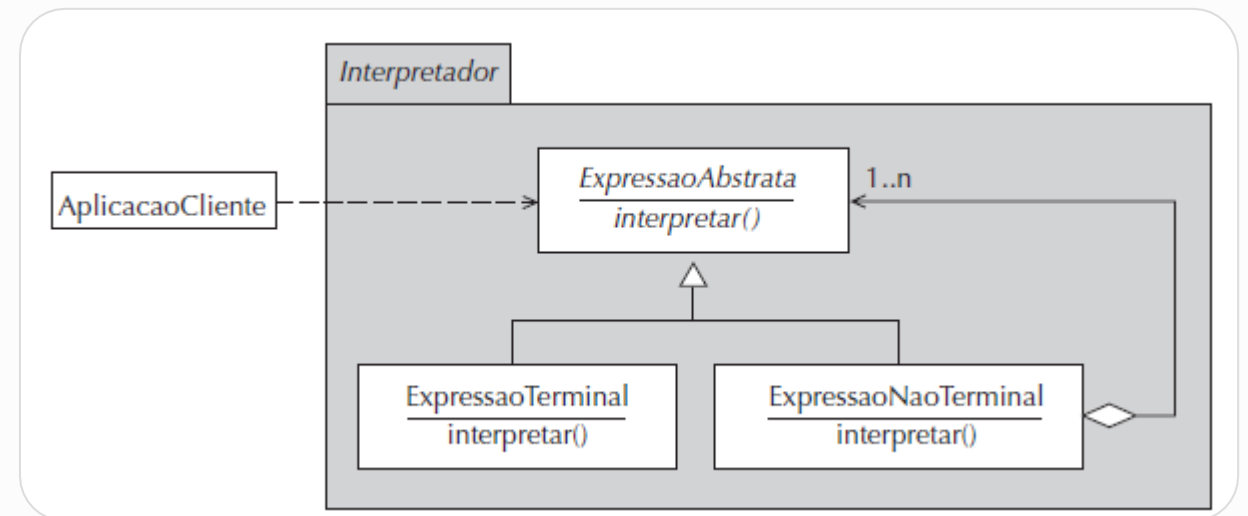
Responsáveis por caracterizar a forma com que as classes e objetos interagem entre si e distribuem as responsabilidades

Padrão de Projeto Interpretador (do inglês Interpreter)

- **Classificação:** Padrão comportamental/Escopo de classes
- **Intenção:** Dada uma linguagem, definir uma representação para sua gramática junto com um interpretador que usa a representação para interpretar sentenças na linguagem.
- **Estrutura:**



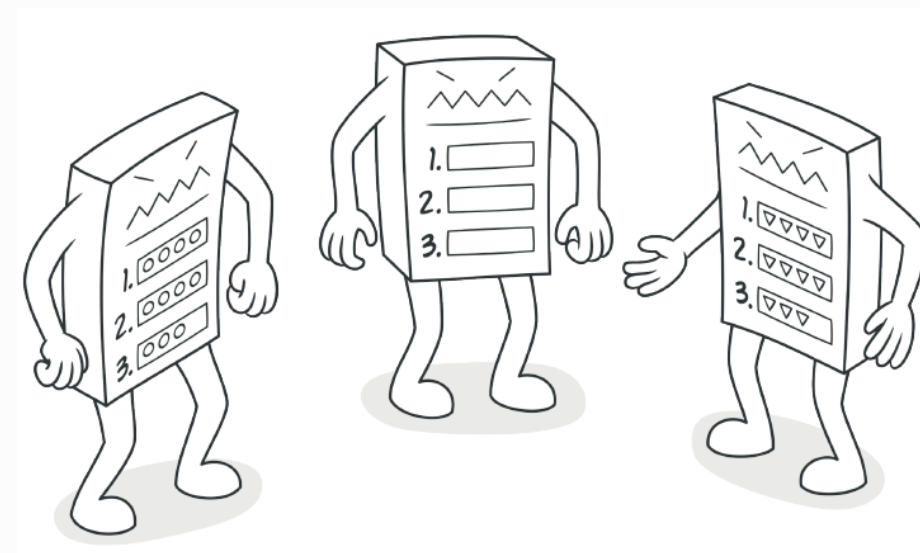
*** Imagem capturada no livro versão html publicado em CD (Gamma et al 1995)*



(Braude, Erick, 2005)

Padrão de Projeto Método-gabarito (do inglês Template Method)

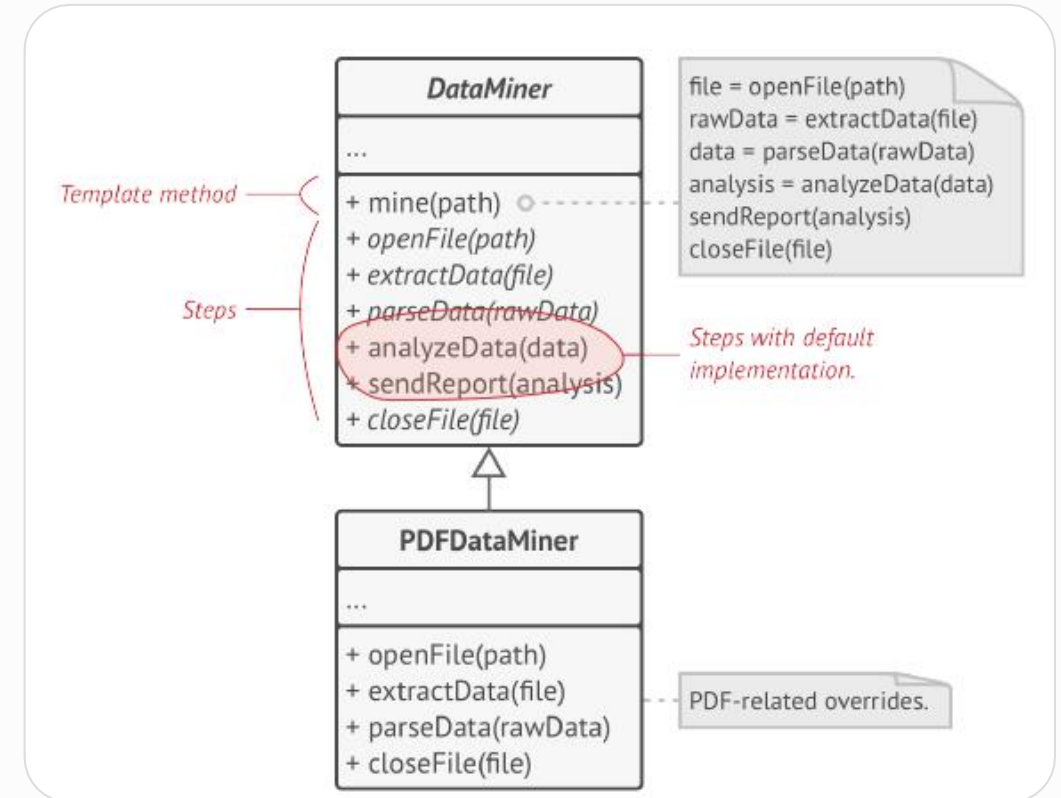
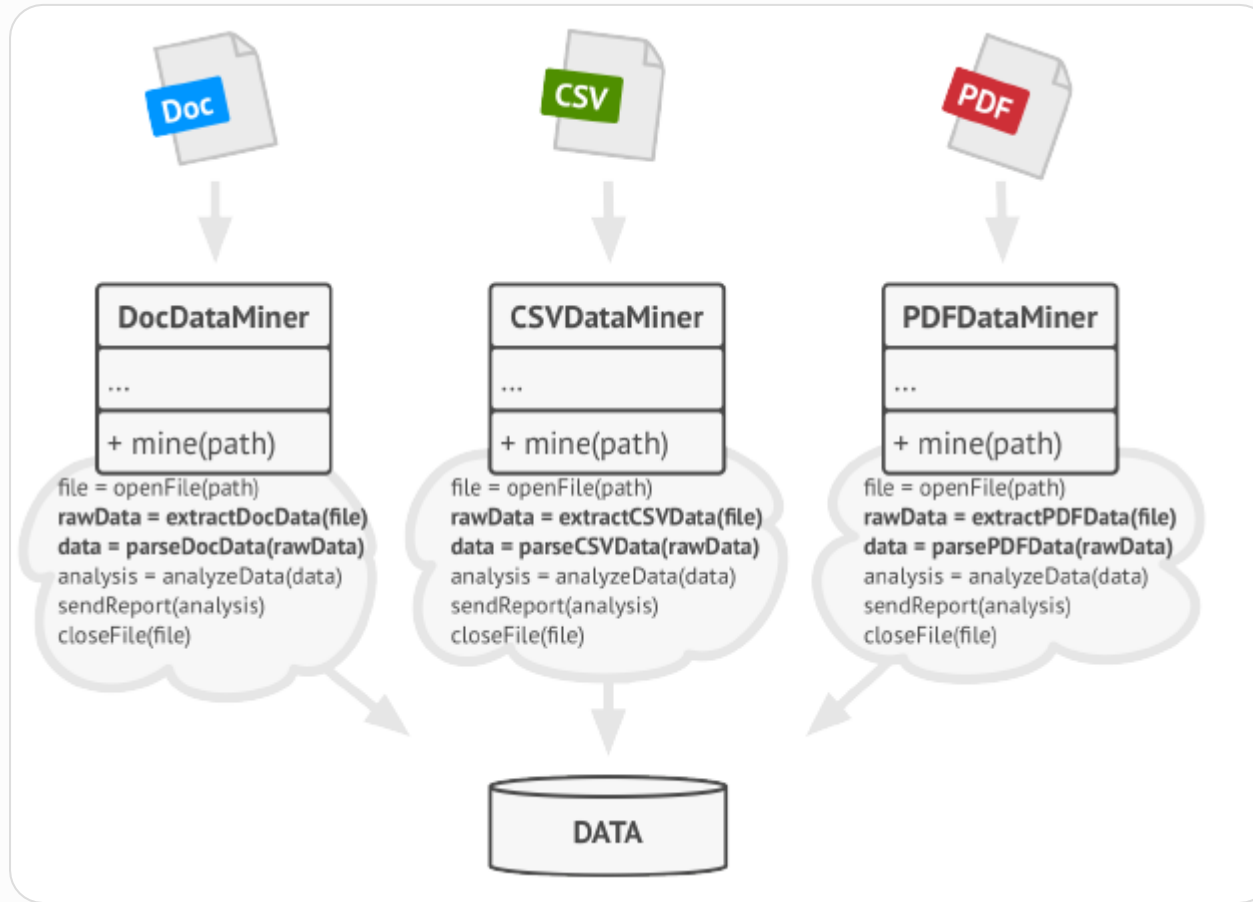
- **Classificação:** Padrão comportamental/Escopo de classes
- **Intenção:** Definir o esqueleto de um algoritmo em uma certa operação, adiando alguns passos para subclasses. O Método-gabarito permite que subclasses redefinam certos passos de um algoritmo sem alterar a estrutura do algoritmo.
 - A ideia do Método-gabarito é definir um método gabarito em uma superclasse, que contenha o esqueleto do algoritmo, com suas partes variáveis e invariáveis. Esse método invoca outros métodos, alguns dos quais são operações que podem ser definidas em uma subclasse.
 - Assim, as subclasses podem redefinir os métodos que variam, acrescentando comportamento específico dos pontos variáveis.



(Shvets, Alexander, 2018)

Padrão de Projeto Método-gabarito

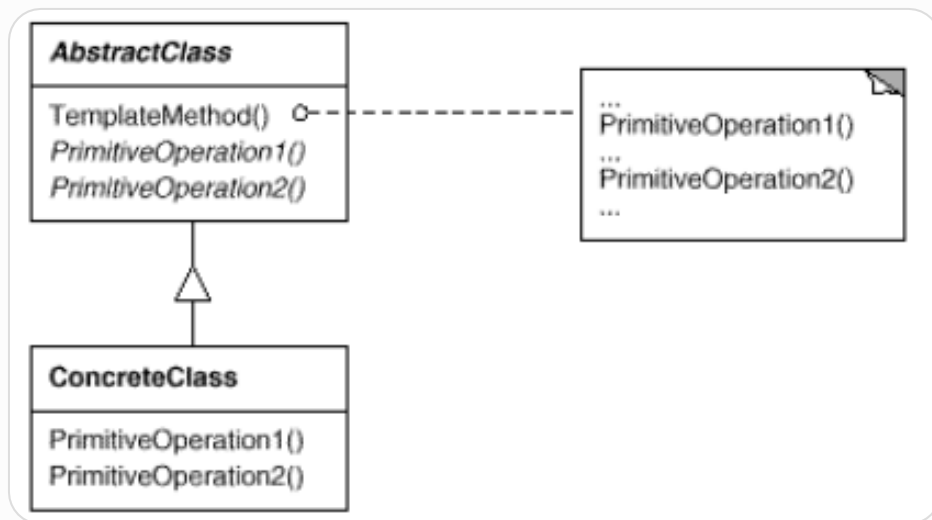
Aplicando o padrão ...



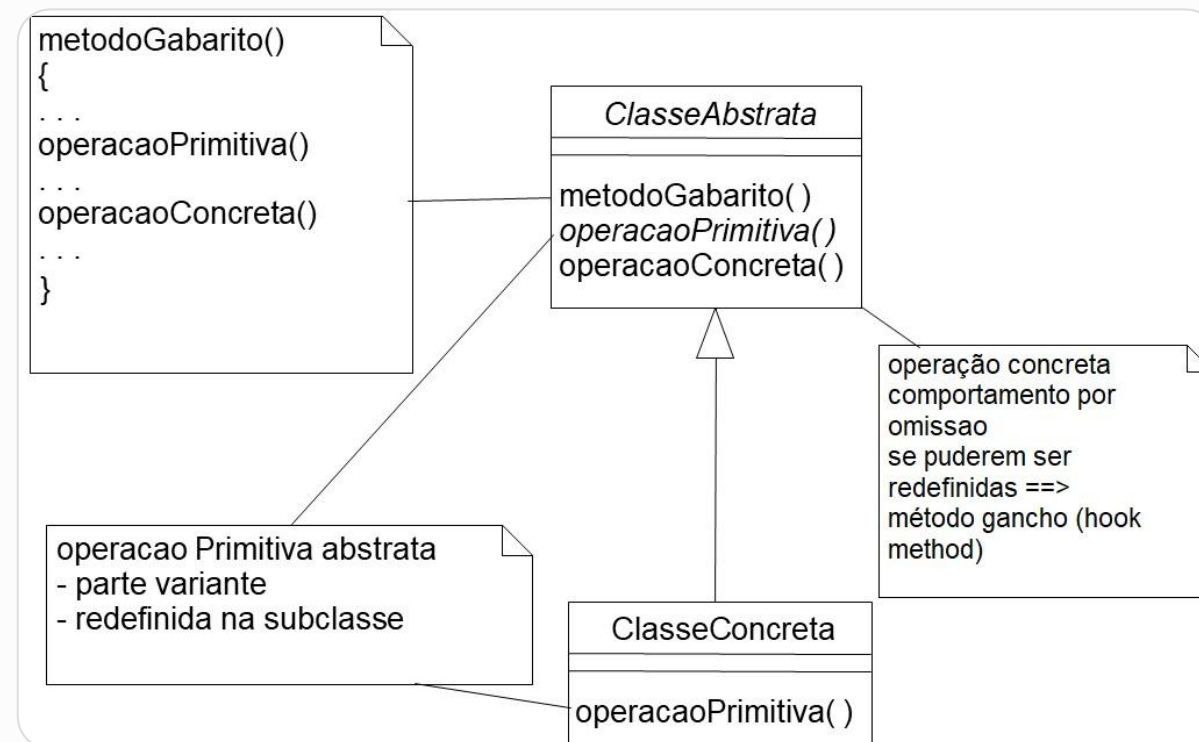
(Shvets, Alexander, 2018)

Padrão de Projeto Método-gabarito

- Estrutura:



*** Imagem capturada no livro versão html
publicado em CD (Gamma et al 1995)*

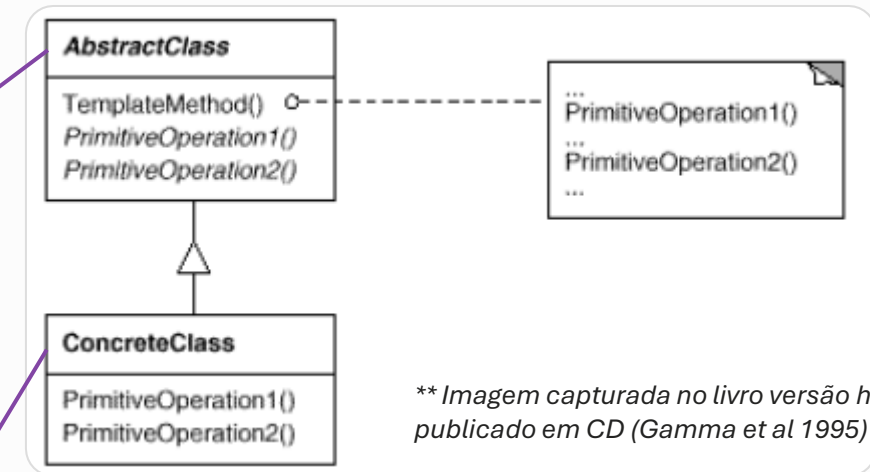


(Braude, 2005)

Padrão de Projeto Método-gabarito

- Participantes:

- define operações primitivas abstratas que subclasses concretas implementam como etapas de um algoritmo.
- implementa um método-gabarito que define o esqueleto de um algoritmo. Ele chama operações primitivas, bem como operações definidas na classe abstrata ou até em outros objetos.



*** Imagem capturada no livro versão html publicado em CD (Gamma et al 1995)*

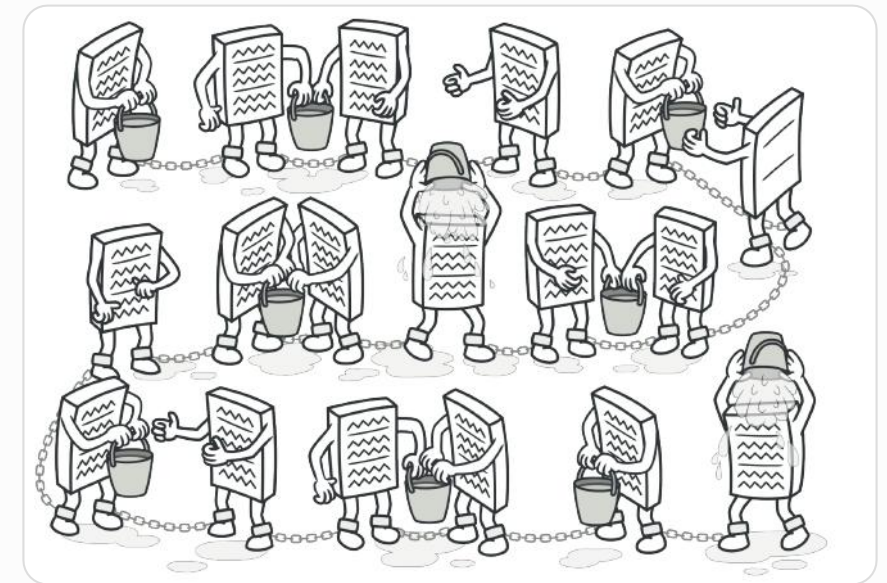
- implementa as operações primitivas para executar etapas específicas da subclasse do algoritmo.

Enquete 8:
Padrão
relacionado?



Padrão de Projeto Cadeia de Responsabilidades (do inglês Chain of Responsibilities)

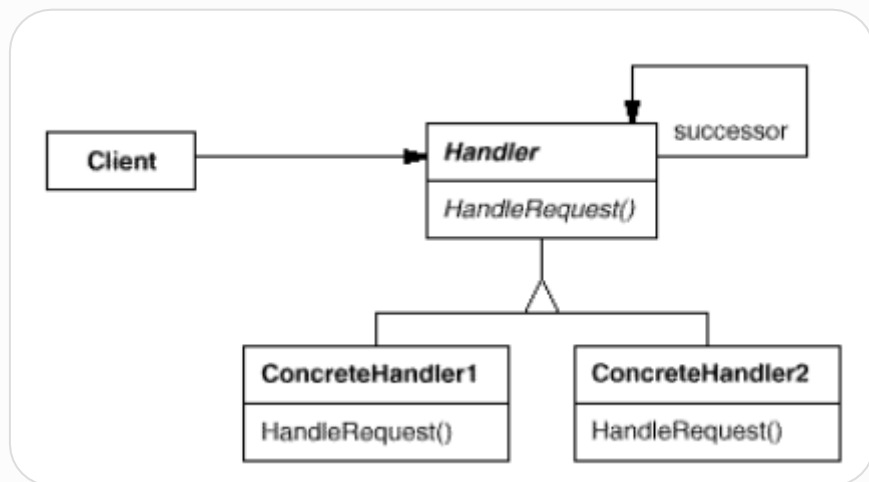
- **Classificação:** Padrão comportamental/Escopo de objetos
- **Intenção:** Evitar acoplar o remetente de uma solicitação ao seu destinatário dando a mais de um objeto a chance de manipular a solicitação. Encadear os objetos receptores e passar a solicitação ao longo da cadeia até que um objeto a manipule.
- **Aplicabilidade:** use o padrão Cadeia de Responsabilidades quando seu programa precisa processar diferentes tipos de requisições de diversas formas, mas os tipos exatos de requisições e sua sequência não são conhecidas antecipadamente



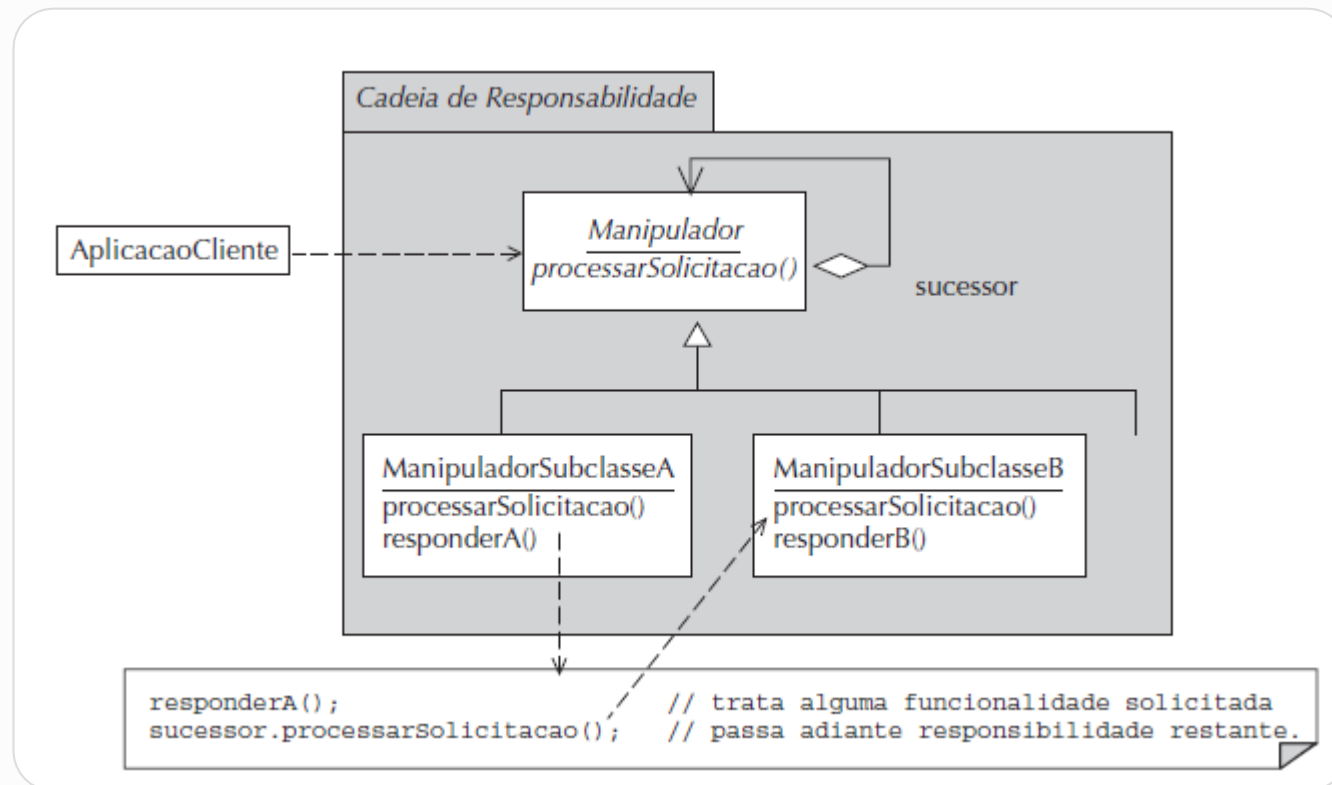
(Shvets, Alexander, 2018)

Padrão de Projeto Cadeia de Responsabilidades

- Estrutura:



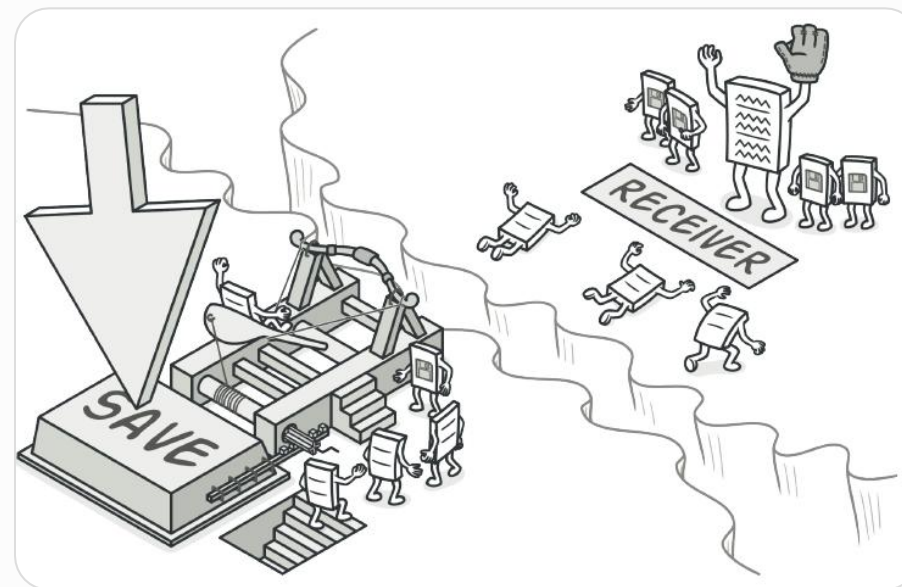
*** Imagem capturada no livro versão html publicado em CD (Gamma et al 1995)*



(Braude, 2005)

Padrão de Projeto Comando (do inglês Command)

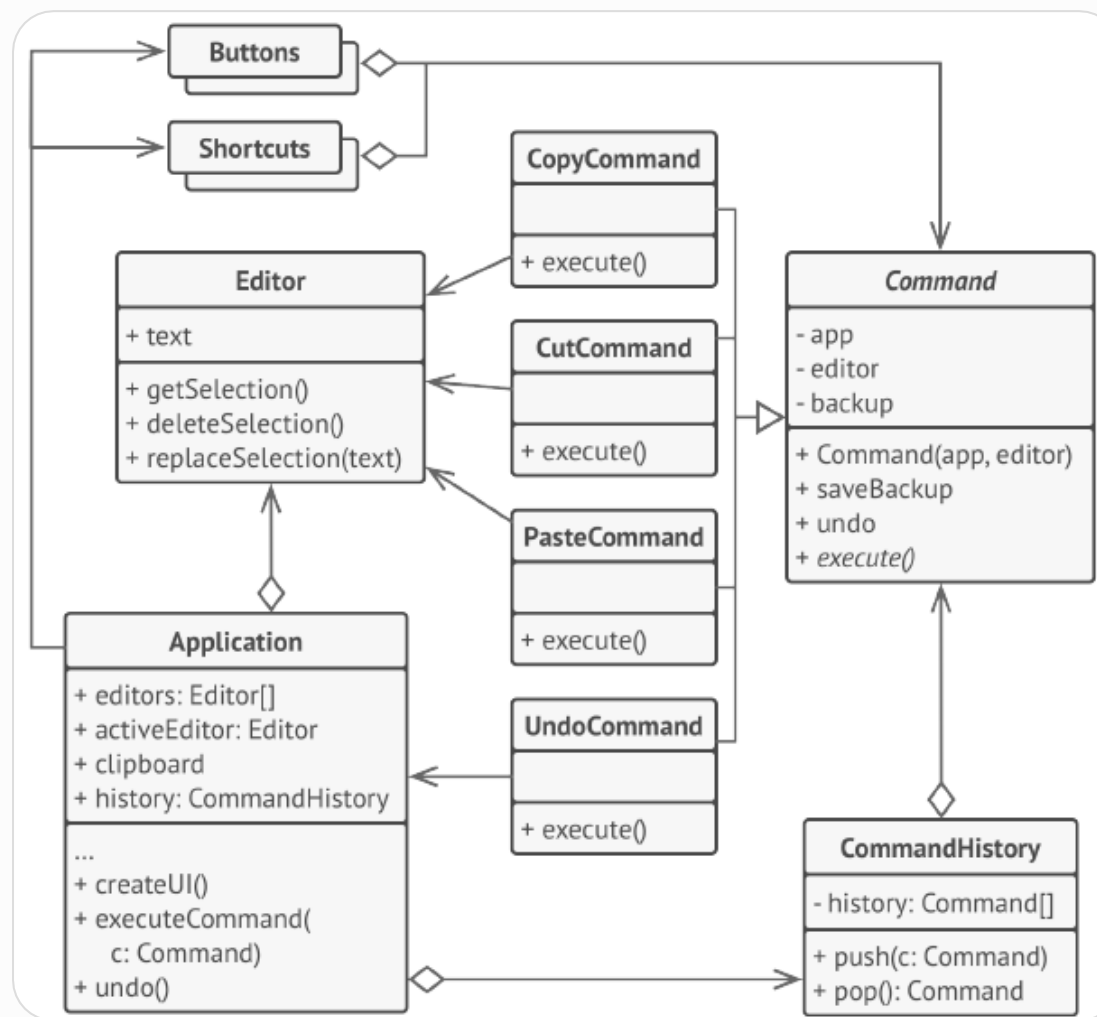
- **Classificação:** Padrão comportamental/Escopo de objetos
- **Intenção:** Encapsular uma solicitação como um objeto, permitindo assim parametrizar os clientes com diferentes solicitações, criar filas ou logs de solicitações e dar suporte a operações que podem ser desfeitas.
- **Motivação:** em algumas aplicações, você pode precisar parametrizar objetos com operações.



(Shvets, Alexander, 2018)

Padrão de Projeto Comando

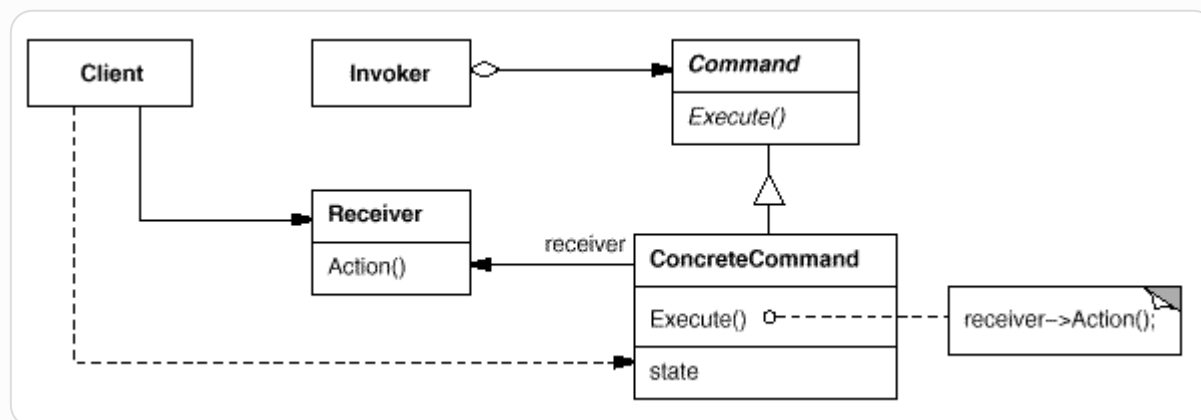
- Exemplo:



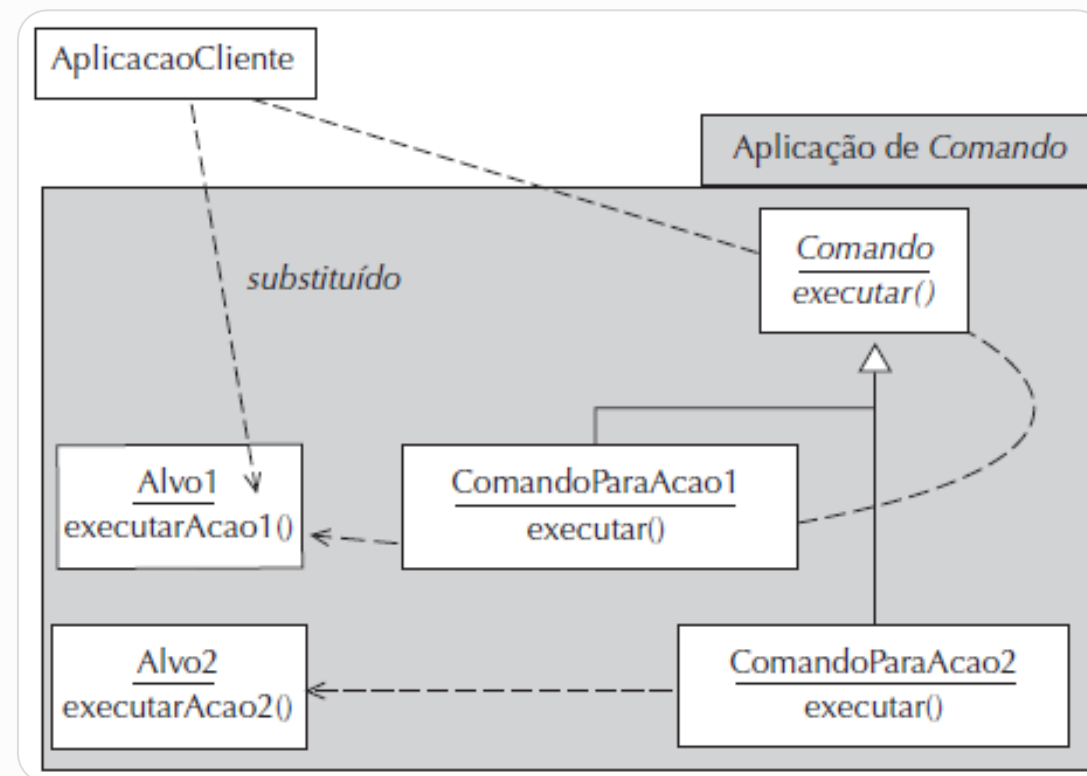
(Shvets, Alexander, 2018)

Padrão de Projeto Comando

- Estrutura:



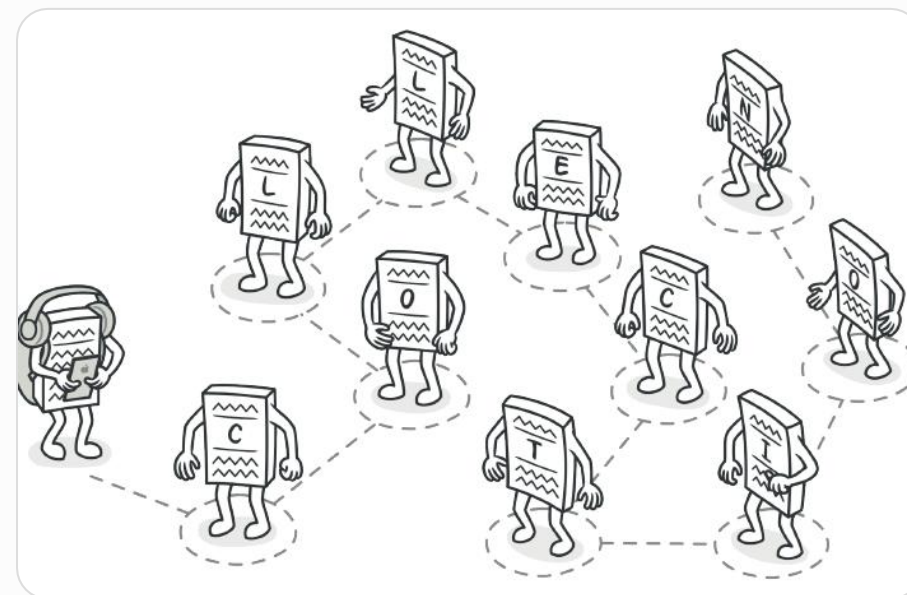
** Imagem capturada no livro versão html
publicado em CD (Gamma et al 1995)



(Braude, 2005)

Padrão de Projeto Iterador (do inglês Iterator)

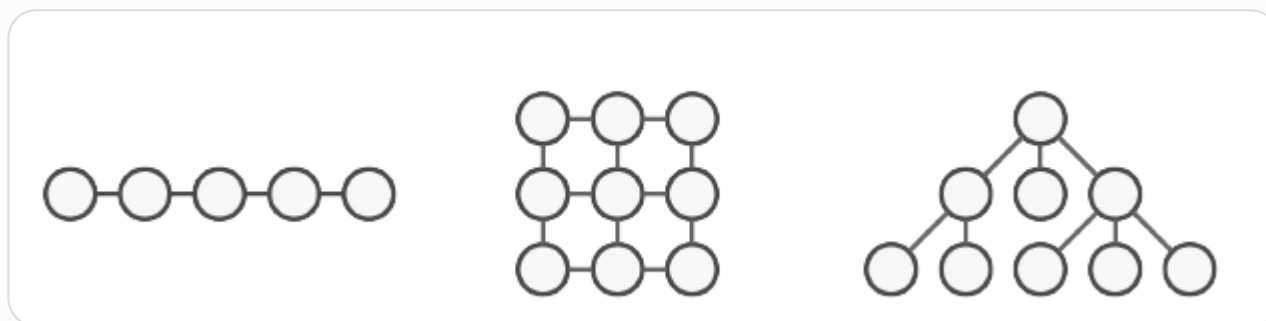
- **Classificação:** Padrão comportamental/Escopo de objetos
- **Intenção:** Fornecer uma maneira de acessar os elementos de um objeto agregado sequencialmente sem expor sua representação subjacente.
- **Motivação:** Existe a necessidade de percorrer agregados quaisquer, em diversas ordens, mas cada um tem uma forma diferente de ser percorrido. Incluir muitos métodos para cuidar da iteração pode poluir a interface. Se for desejável percorrer o agregado de diferentes formas, ainda mais métodos serão necessários.



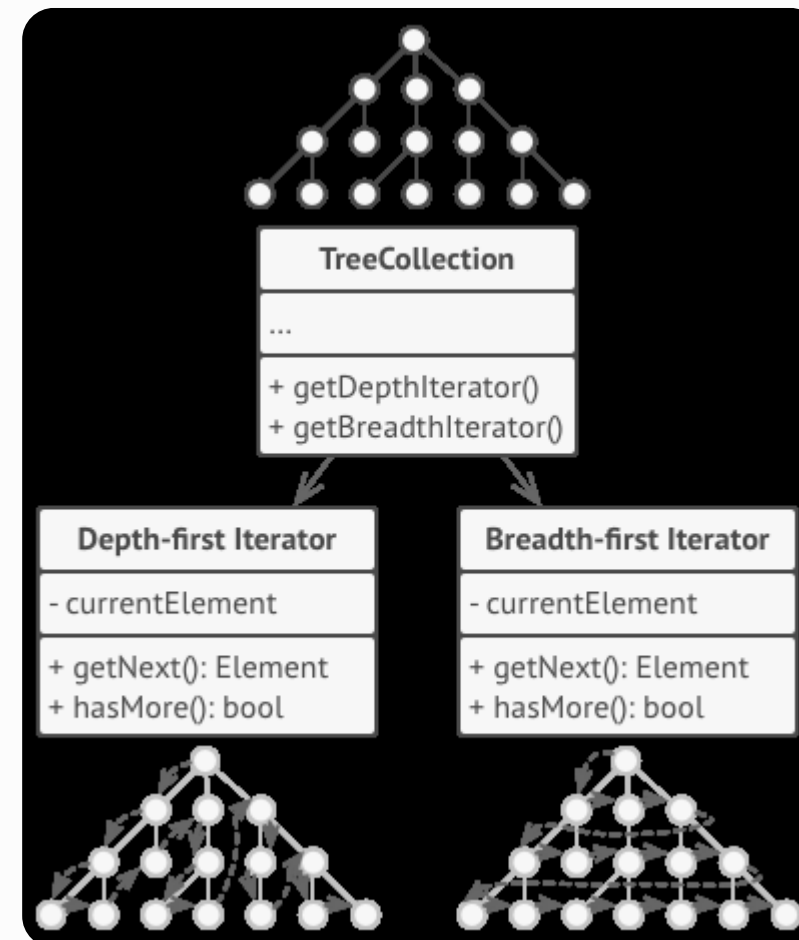
(Shvets, Alexander, 2018)

Padrão de Projeto Iterador

- **Aplicabilidade:**
- Use o padrão Iterador para:
 - acessar o conteúdo de um objeto agregado sem expor sua representação interna.
 - dar suporte a múltiplos trajetos de objetos agregados.
 - fornecer uma interface uniforme para percorrer diferentes estruturas agregadas (isto é, para apoiar iteração polimórfica)

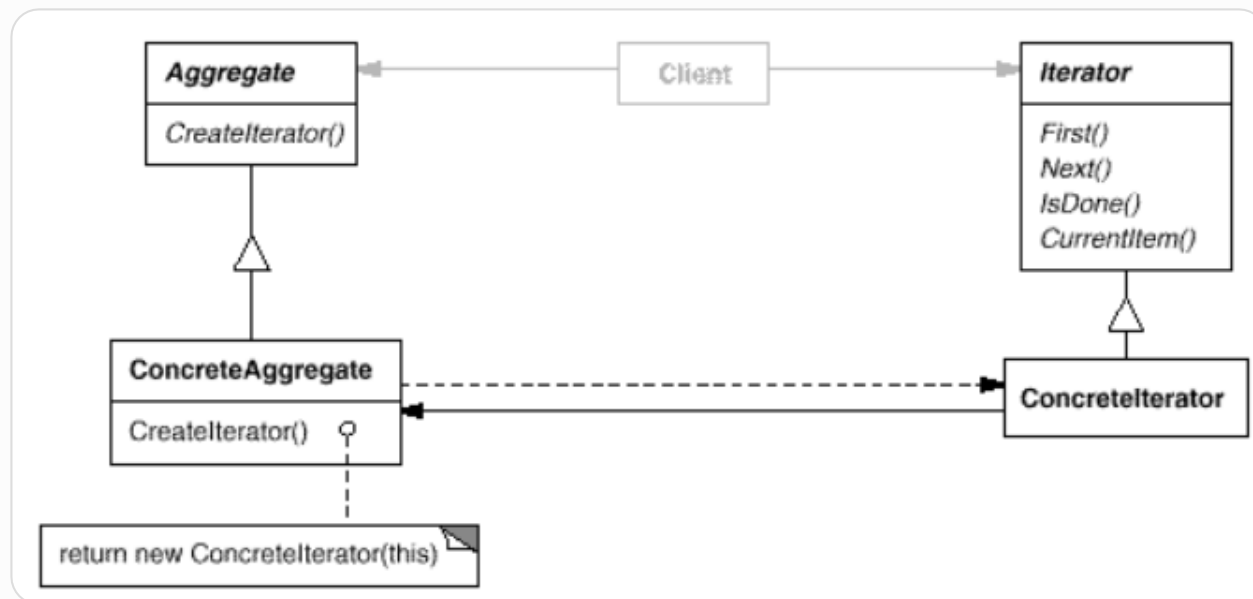


(Shvets, Alexander, 2018)

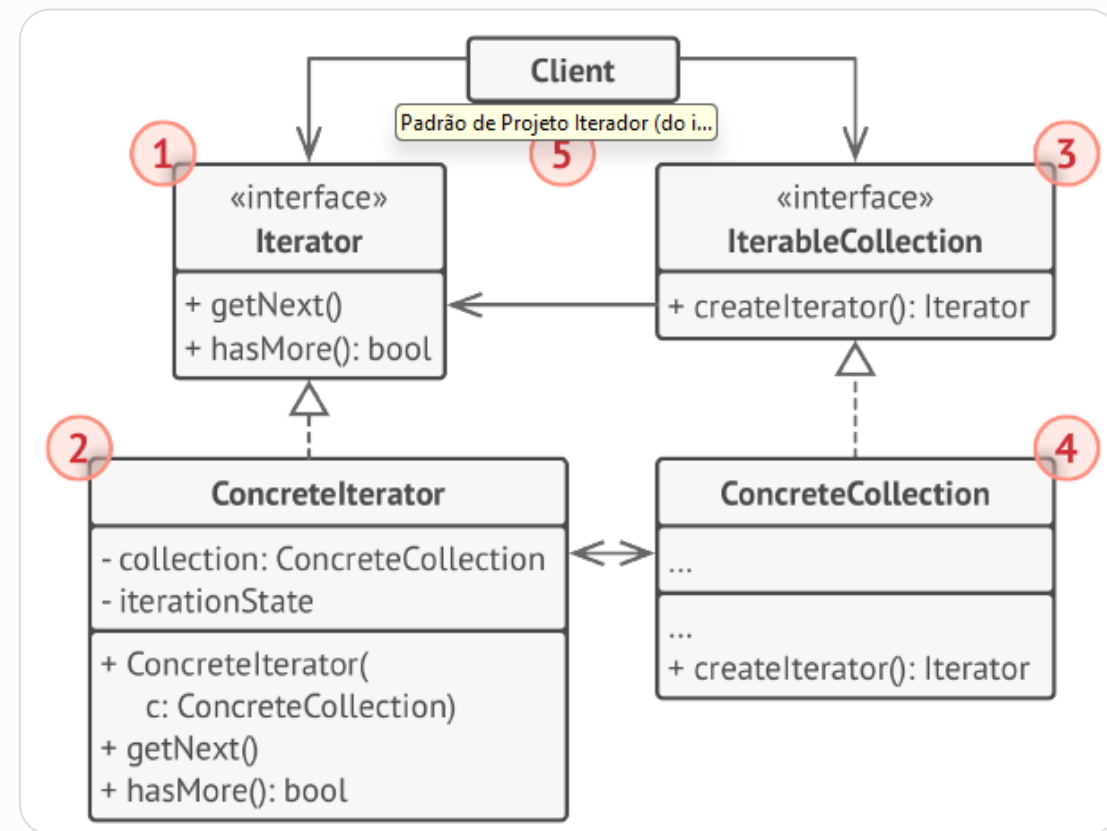


Padrão de Projeto Iterador

- Estrutura:



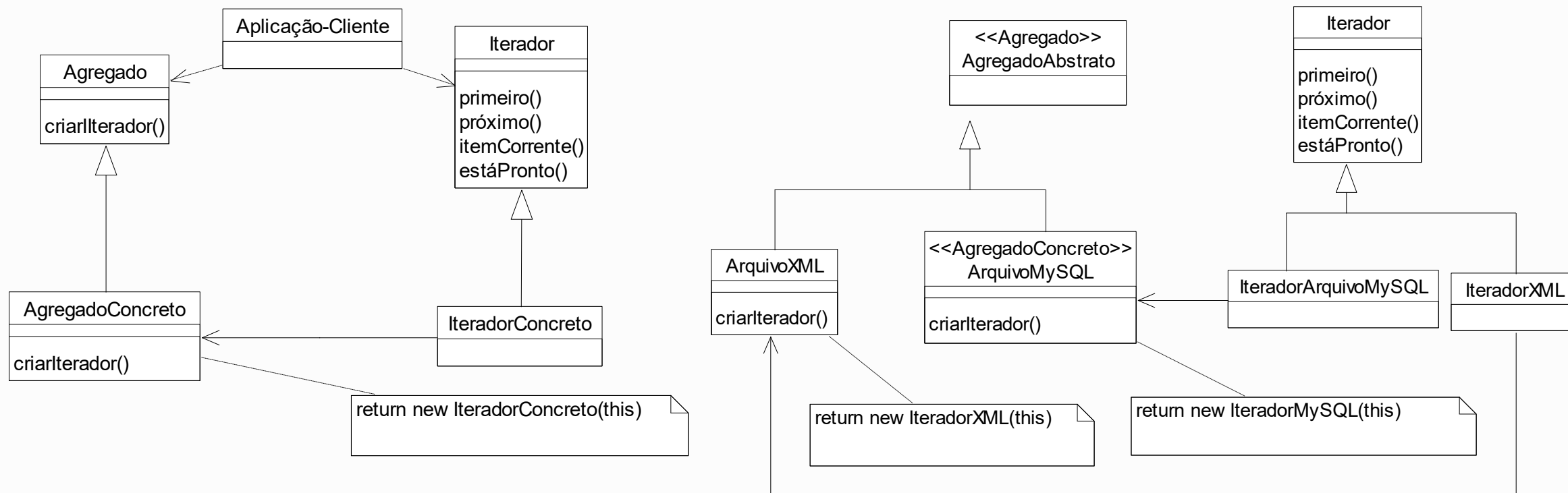
*** Imagem capturada no livro versão html publicado em CD (Gamma et al 1995)*



(Shvets, Alexander, 2018)

Padrão de Projeto Iterador

- Exemplo:

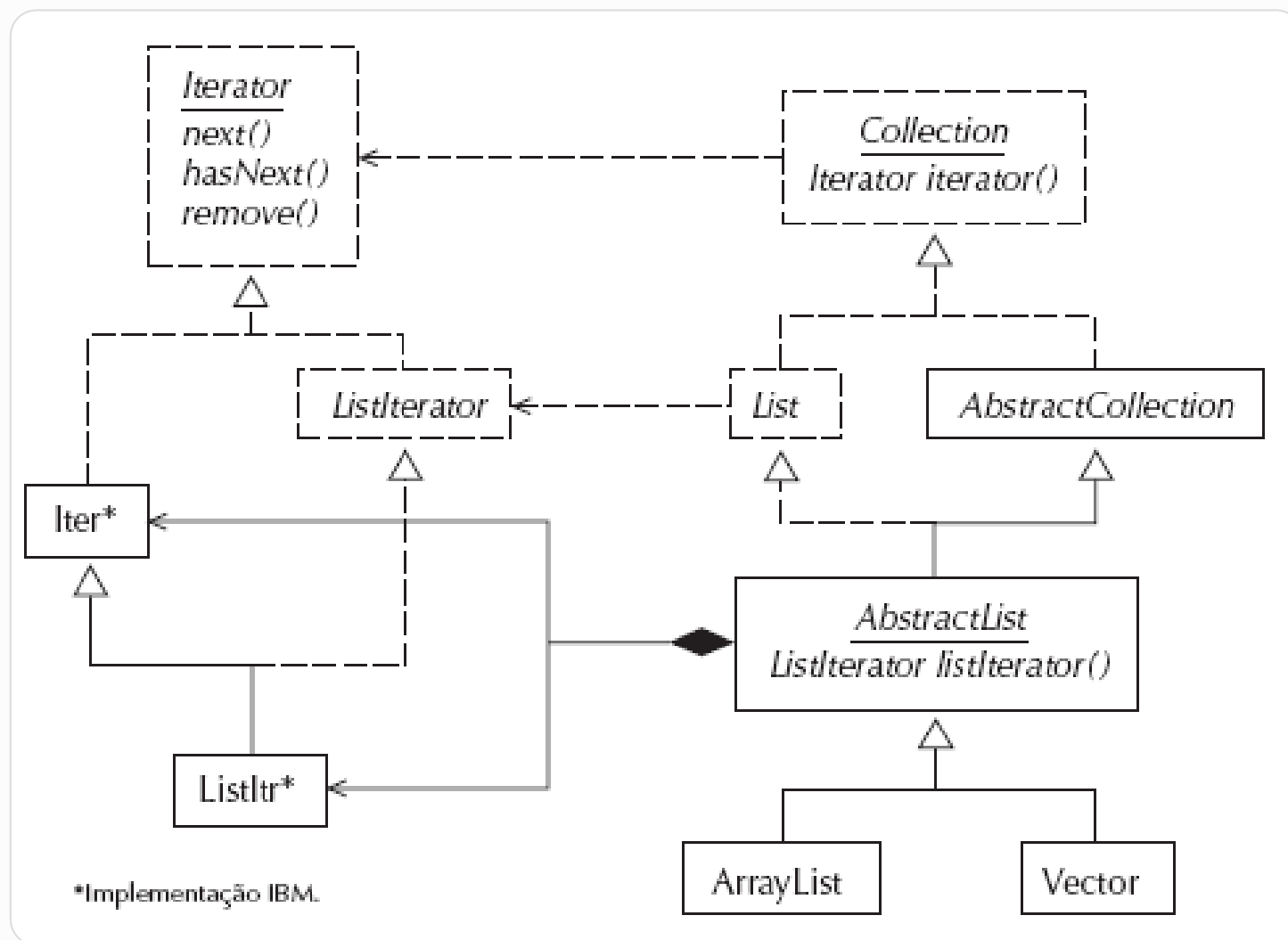


(Braude, 2005)

Padrão de Projeto Iterador

- Iterador na API Java
 - O pacote `java.util` contém uma interface `Iterator` com os métodos: `next()`, `hasNext()` e `remove()`.
 - O método `next()` é uma combinação de `proximo()` e `itemCorrente()`
 - A subinterface `ListIterator` especifica esses métodos para operarem nos objetos `List` e adiciona os métodos apropriados aos objetos `List`.
 - A interface `List` tem um método `listIterator()`, que retorna um objeto `ListIterator` que itera sobre ele. Isso permite ao programador mover-se e iterar sobre objetos `List`.

Padrão de Projeto Iterador

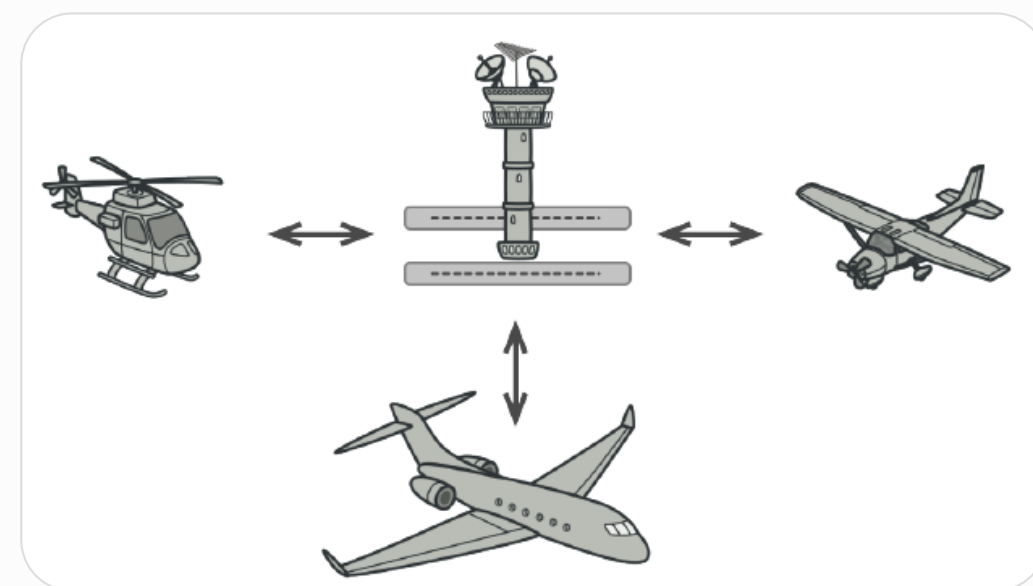


Padrão de Projeto Iterador

```
public class Leitor
{
    private String nome;
    private Char[] tipo;
    private Boolean achou=false;
    private List emprestimos = new ArrayList();
    public void devolverCopia(int codCopia)
    {
        Iterator i = emprestimos.iterator();
        while (i.hasNext()) && (!achou) {
            Emprestimo e = (Emprestimo) i.next();
            achou=e.devolverCopia(codCopia) }
        }
    }
```

Padrão de Projeto Mediador (do inglês Mediator)

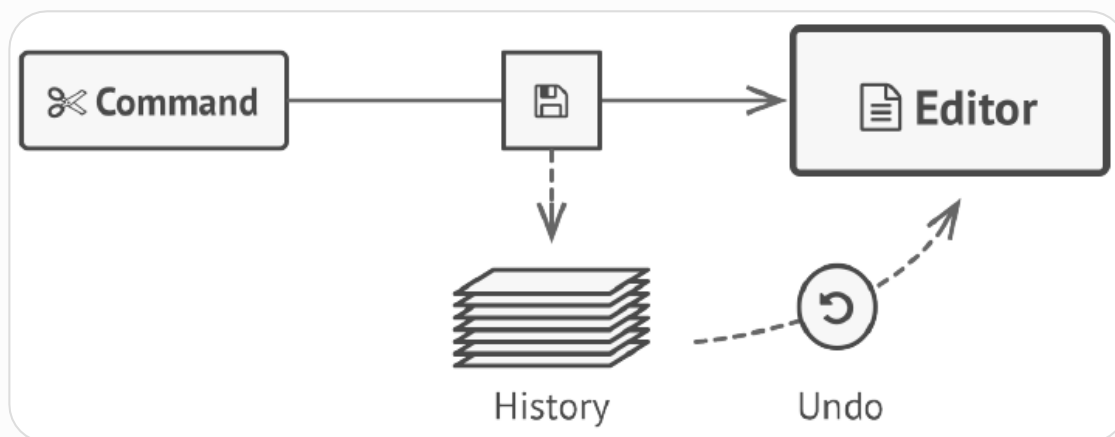
- **Classificação:** Padrão comportamental/Escopo de objetos
- **Intenção:** Definir um objeto que encapsula a forma como um conjunto de objetos interage. O Mediador promove acoplamento fraco ao impedir que objetos se refiram uns aos outros explicitamente, e permite que você varie a interação deles independentemente.



(Shvets, Alexander, 2018)

Padrão de Projeto Memento

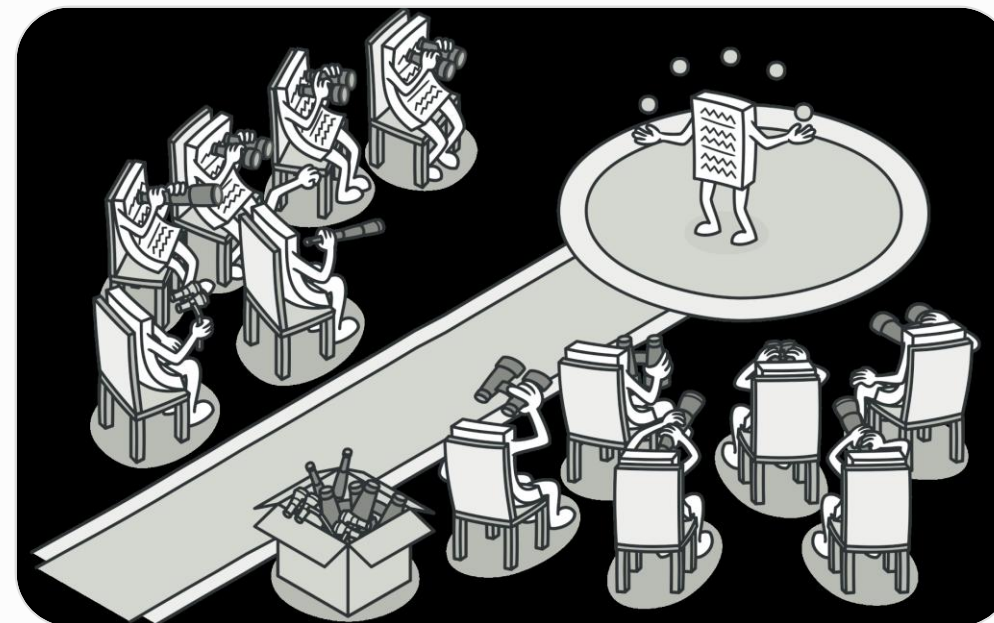
- **Classificação:** Padrão comportamental/Esopo de objetos
- **Intenção:** Sem violar o encapsulamento, capturar e externalizar o estado interno de um objeto para que o objeto possa ser restaurado a esse estado posteriormente.



(Shvets, Alexander, 2018)

Padrão de Projeto Observador (em inglês Observer)

- **Classificação:** Padrão comportamental/Escopo de objetos
- **Intenção:** Definir uma dependência de um para muitos entre objetos para que, quando um objeto mudar de estado, todos os seus dependentes sejam notificados e atualizados automaticamente.

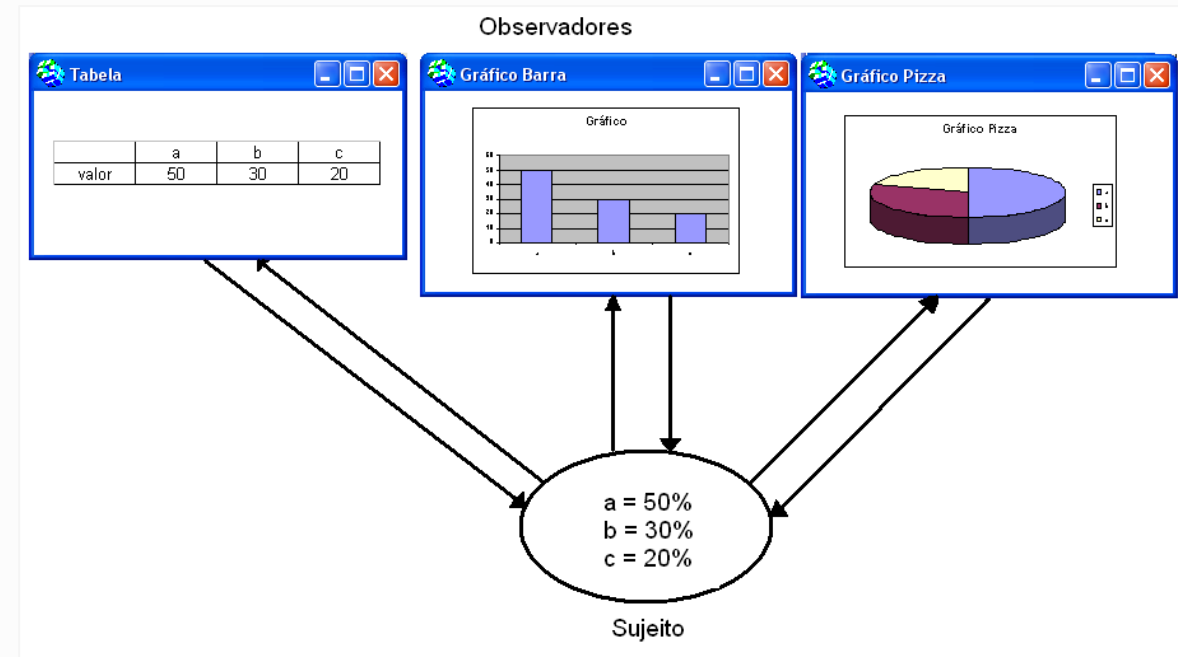


(Shvets, Alexander, 2018)

Padrão de Projeto Observador

- **Aplicabilidade:**

- sistemas cujas informações são preferencialmente centralizadas em uma única fonte (sujeito), e existem diversas visões (observadores) que devem ser atualizadas sempre que a informação for modificada
 - Qualquer tentativa de manter a consistência aumenta o acoplamento entre as classes.
 - Por exemplo, em uma ferramenta integrada, pode-se separar os dados em si das diversas visões sobre esses dados, de forma que mudança em uma das visões seja refletida nas demais, mesmo que as visões não tenham conhecimento umas das outras.
- Quando um objeto deve ser capaz de notificar outros objetos sem fazer suposições sobre quem são esses objetos, ou não os conhecendo de antemão ou quando esses objetos observadores mudam dinamicamente



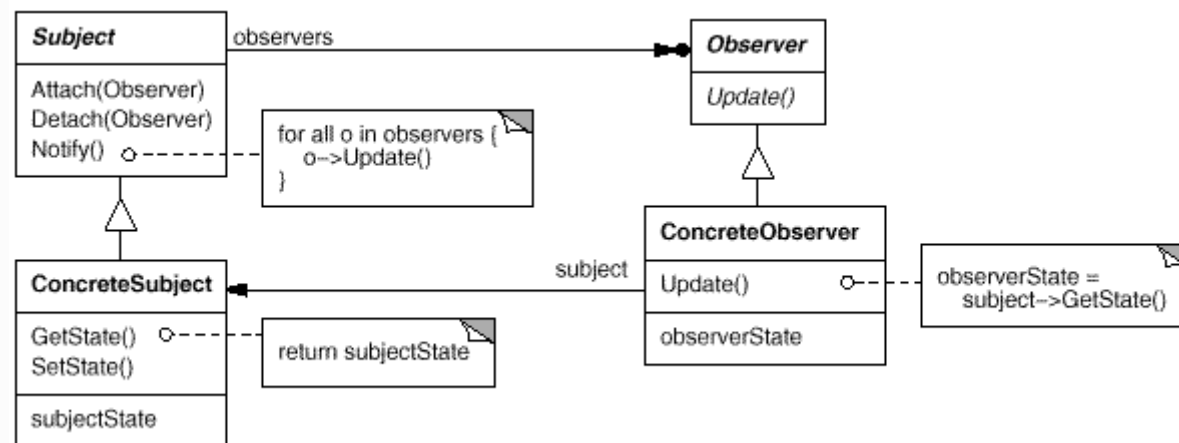
Enquete 9:

O que isso te lembra?

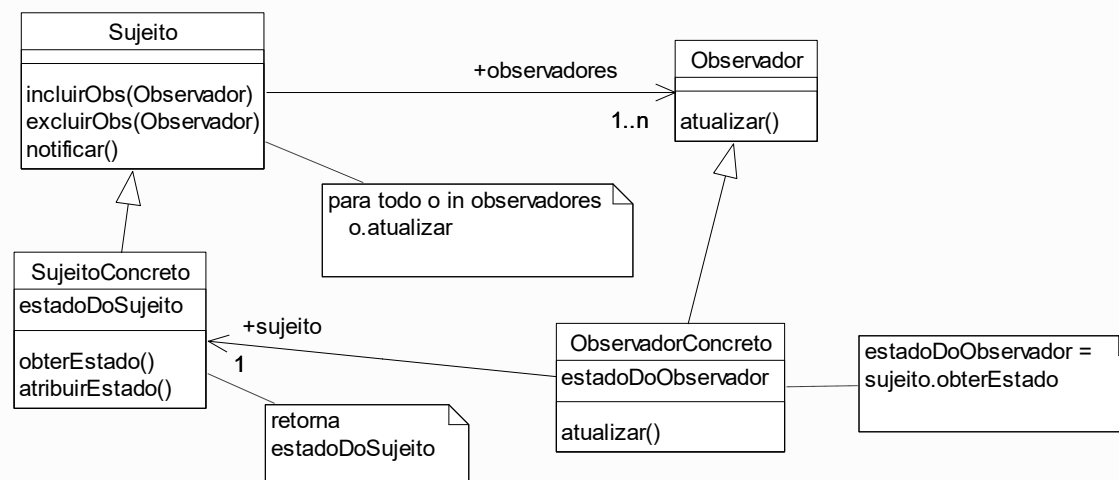


Padrão de Projeto Observador

- Estrutura:



*** Imagem capturada no livro versão html publicado em CD (Gamma et al 1995)*

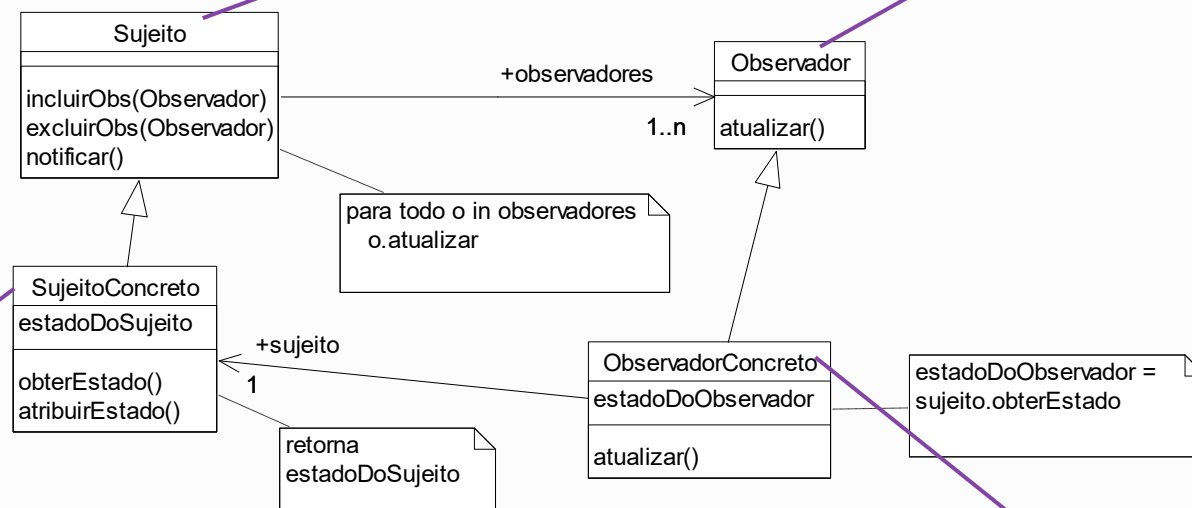


Padrão de Projeto Observador

- **Participantes:**

Conhece seus observadores, provê uma interface para incluir e exclui-los

Define uma interface de atualização para objetos que precisarão ser notificados de mudanças no sujeito

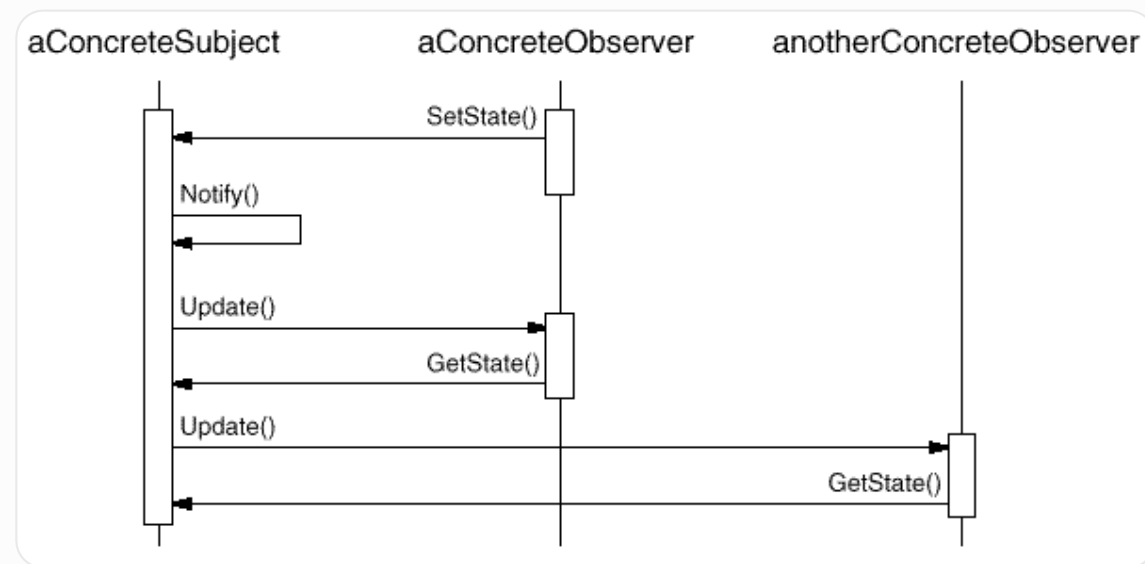


- Armazena o estado de interesse para objetos **ObservadorConcreto**.
- Envia uma notificação para seus observadores quando seu estado muda

- Mantem referência para o objeto **SujeitoConcreto**
- Armazena o estado que deve ser consistente com o sujeito
 - Implementa o método abstrato `atualizar`

Padrão de Projeto Observador

- Formas de implementação:
 - Os próprios observadores podem enviar atualizações de estado para o sujeito concreto, que por sua vez notifica a todos os observadores
 - Os demais observadores podem obter o estado para saber se precisam se atualizar ou não
 - Outra forma de implementação seria os observadores, ao se inscrever, especificar algumas condições para que só recebam a notificação caso as condições sejam verdadeiras
 - No livro de Gamma há várias outras considerações importantes para levar em conta ao implementar esse padrão



*** Imagem capturada no livro versão html
publicado em CD (Gamma et al 1995)*

Padrão de Projeto Observador

- **Exemplo:**
- projetar a interface gráfica com o usuário para um sistema de agendamentos atribuídos aos diversos veterinários que trabalham em uma clínica.
- Na visão da secretária, devem aparecer todos os agendamentos
- Na visão de cada veterinário, aparece somente sua agenda particular

Clínica Veterinária X
Agenda do dia 22/04/2006

8:00	Mariana/Lulu (Dr. João Carlos)
	Carlos/Beethoven (Dra. Maria Helena)
9:00	_____

10:00	Jessica/Tobias (Dra. Maria Helena)

Clínica Veterinária X
Agenda do dia 22/04/2006
Dr. João Carlos

8:00	Mariana/Lulu
9:00	_____

10:00	_____

11:00	Fátima/Rex

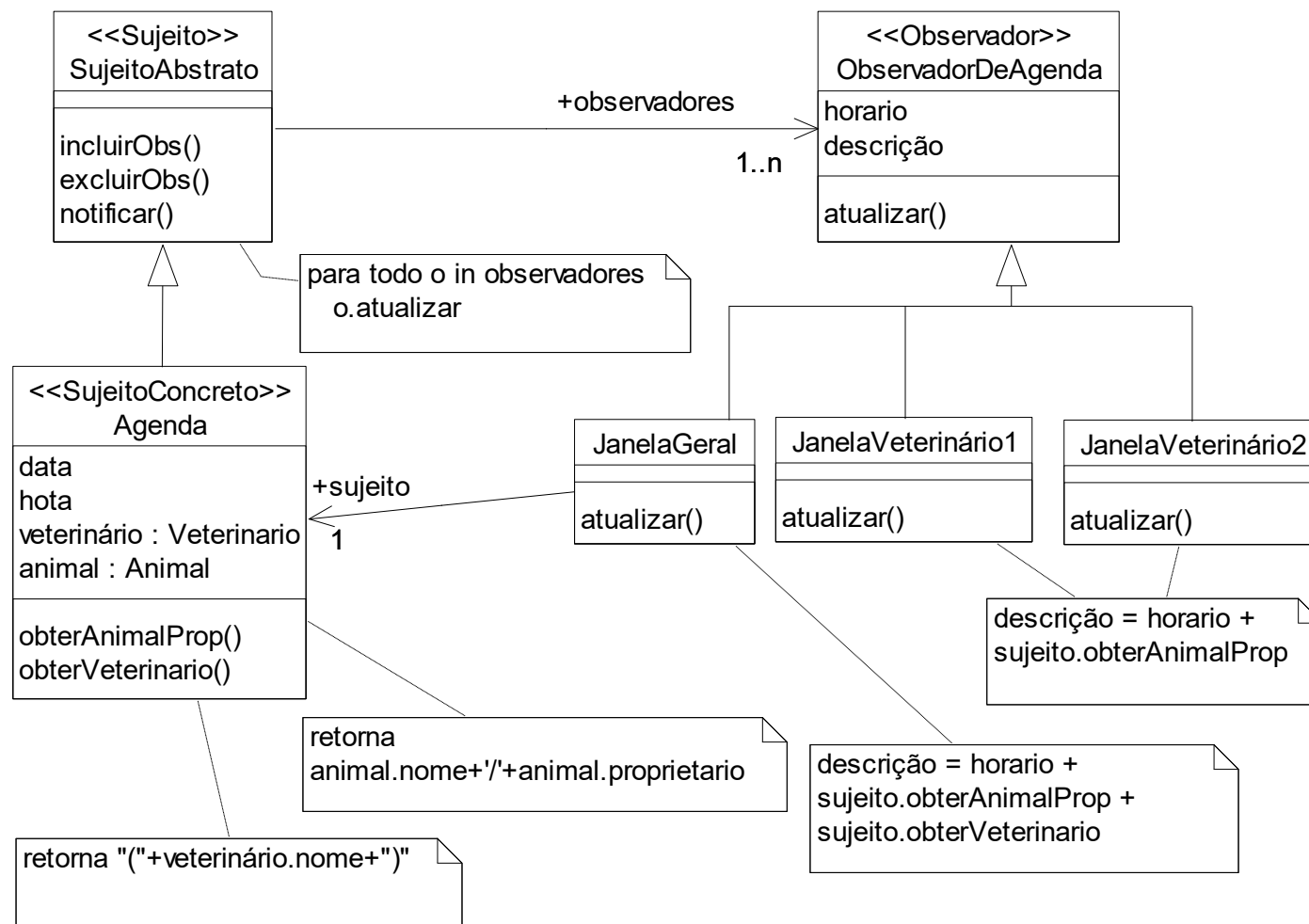
Clínica Veterinária X
Agenda do dia 22/04/2006
Dra. Maria Helena

8:00	Carlos/Beethoven
9:00	_____

10:00	Jessica/Tobias

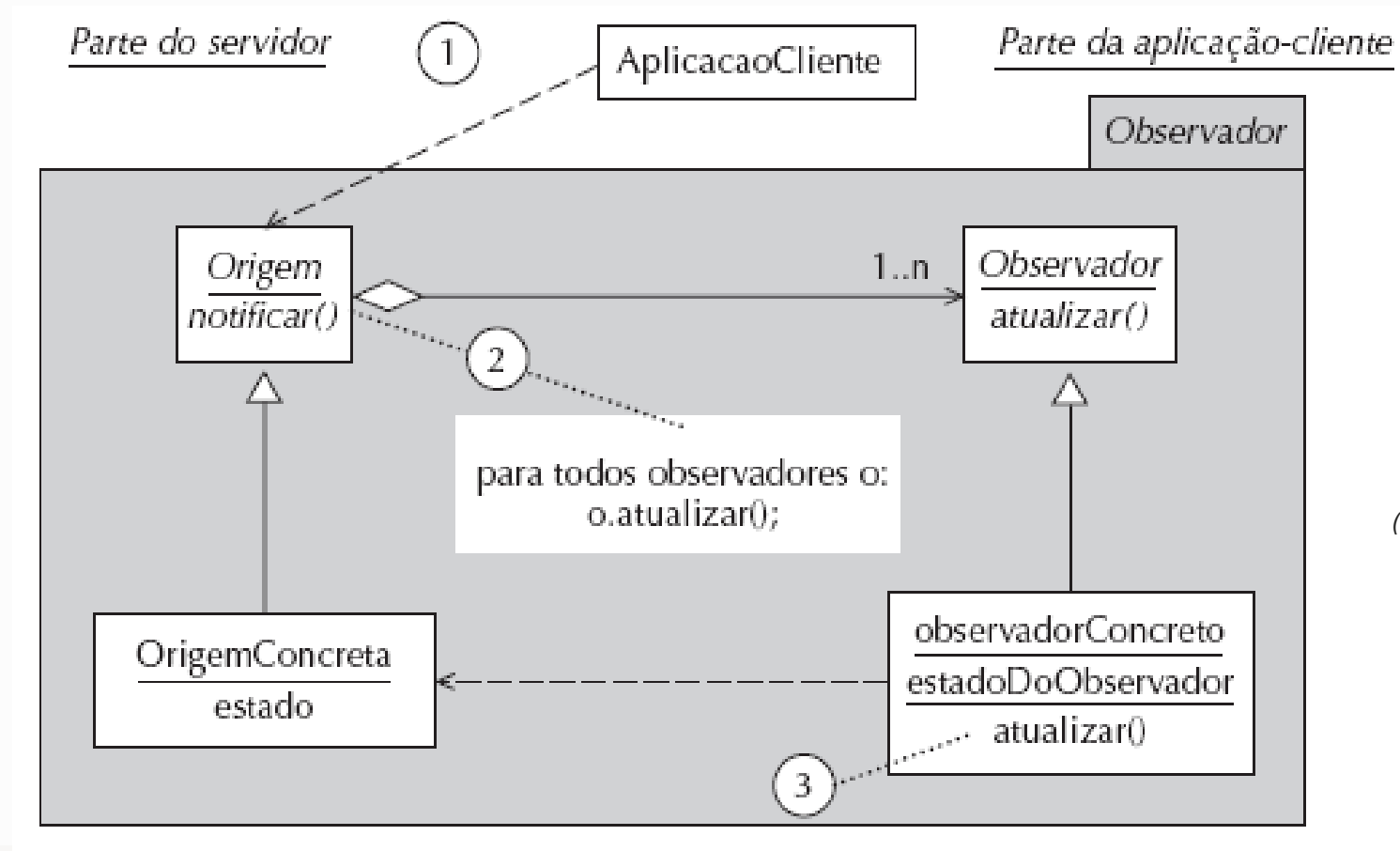
11:00	_____

Padrão de Projeto Observador



Padrão de Projeto Observador

- Como funciona o Observador?

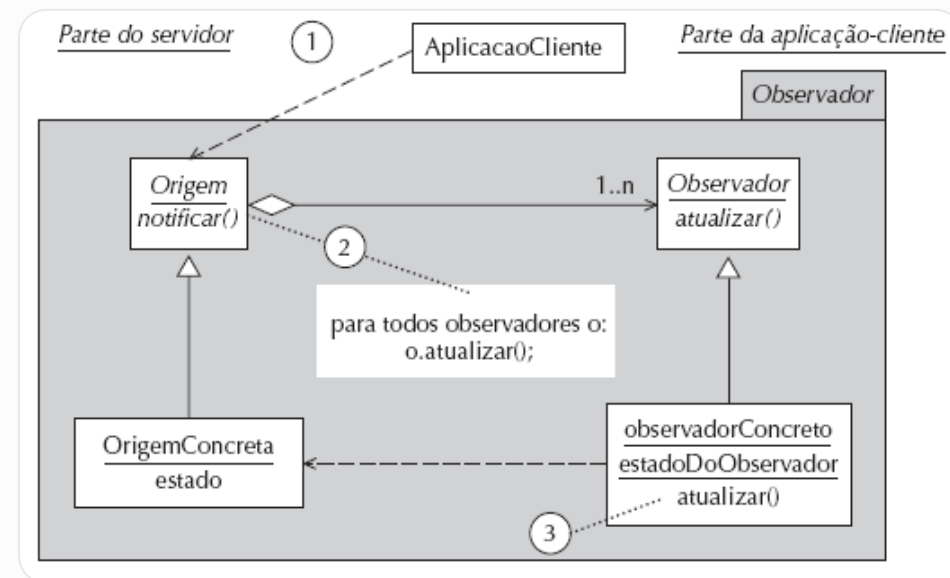


(Braude, 2005)

Padrão de Projeto Observador

- Como funciona?

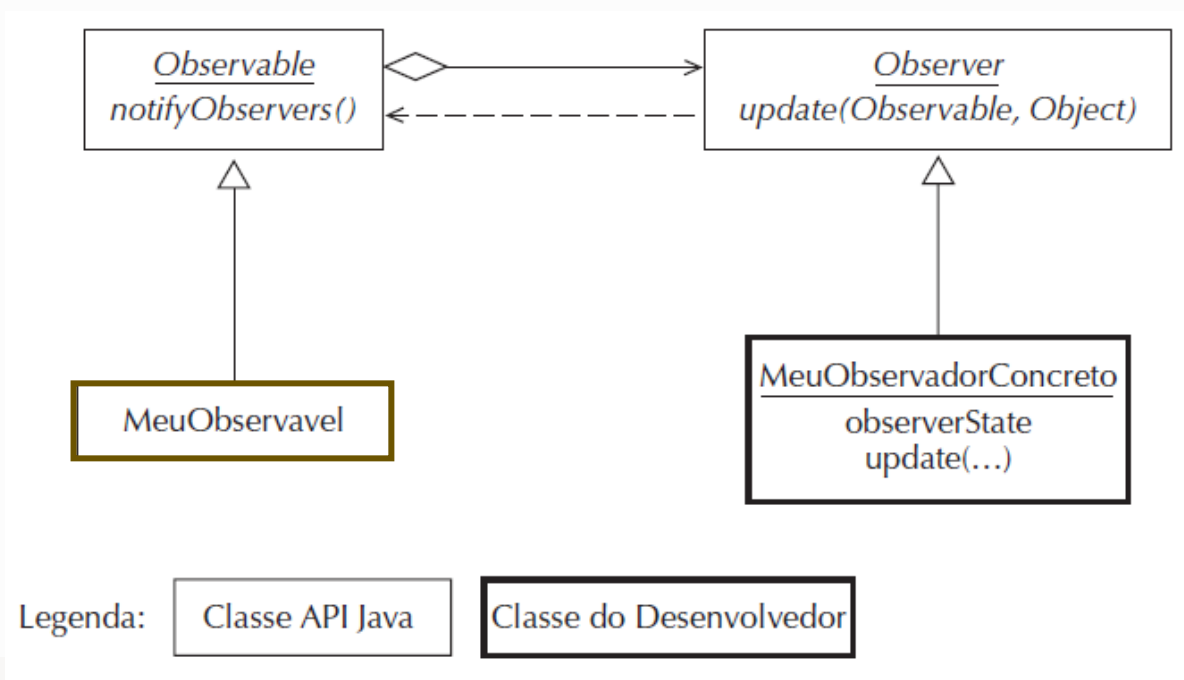
- (Passo 1) A aplicação cliente referencia um objeto de interface conhecido, solicitando que os observadores sejam notificados.
 - Por exemplo, a aplicação cliente poderia ser um processo programado para alertar sobre uma alteração de dados. No modelo, isso é mostrado como um objeto AplicacaoCliente, que informa ao objeto Origem para executar sua função notificar().
- (Passo 2) O método notificar() chama a função atualizar() em cada objeto Observador que ele agrega.
- (Passo 3) A implementação de atualizar() depende do ObservadorConcreto particular a que pertence.
 - Normalmente, atualizar() compara o estado do objeto ObservadorConcreto (valores de variáveis) com aquele da origem de dados central, para então decidir se deve ou não alterar seus valores de variáveis da mesma forma.



(Braude, 2005)

Padrão de Projeto Observador

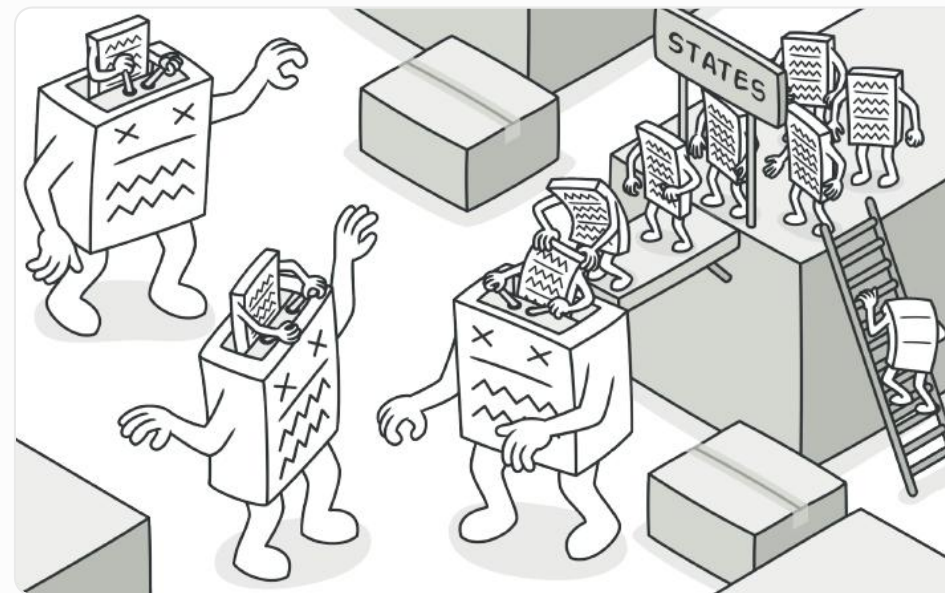
- Vale lembrar que esse padrão existe na API Java
 - Observe que `update()` é um método call-back, porque fornece aos objetos `Observer` uma referência a sua origem, permitindo que comparem seus dados, por exemplo com o objeto `Observable` ao executar o `update`.
 - Como a atualização é feita de modo retroativo, não há necessidade das classes concretas `Observer` manterem referências ao objeto `Observable`.



(Braude, 2005)

Padrão de Projeto Estado (do inglês State)

- **Classificação:** Padrão comportamental/Escopo de objetos
- **Intenção:** Permitir que um objeto altere seu comportamento quando seu estado interno mudar. A ideia é fazer parecer que o objeto mudou de classe.
- **Aplicabilidade:**
 - quando o comportamento de um objeto depende de seu estado, que pode mudar em tempo de execução
 - quando essa mudança de comportamento implementada com o uso de estruturas case favorece erros, principalmente quando o sistema evolui e novos estados surgem, pois várias operações conterão essas estruturas case.

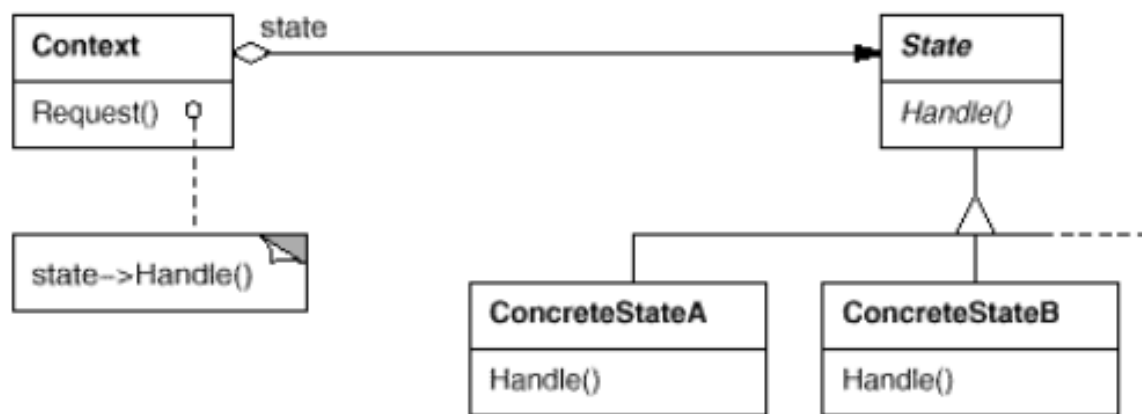


(Shvets, Alexander, 2018)

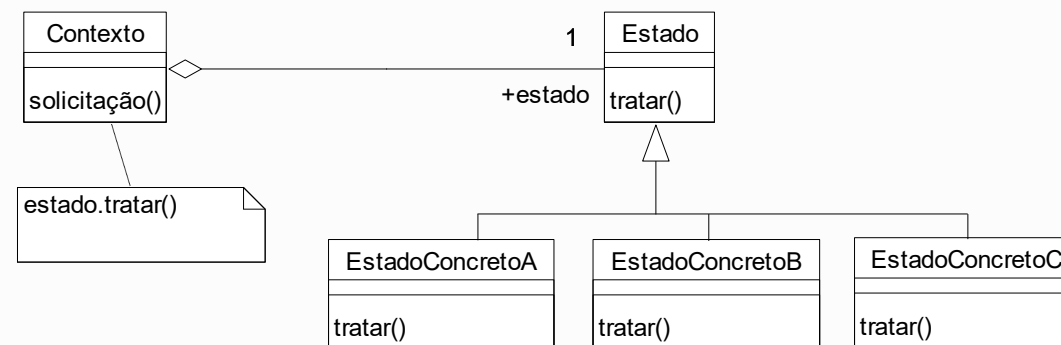
Padrão de Projeto Estado

- Estrutura:

- Criar uma classe para representar o objeto em si e outra para representar seu estado
- O estado pode variar independentemente de outros objetos.



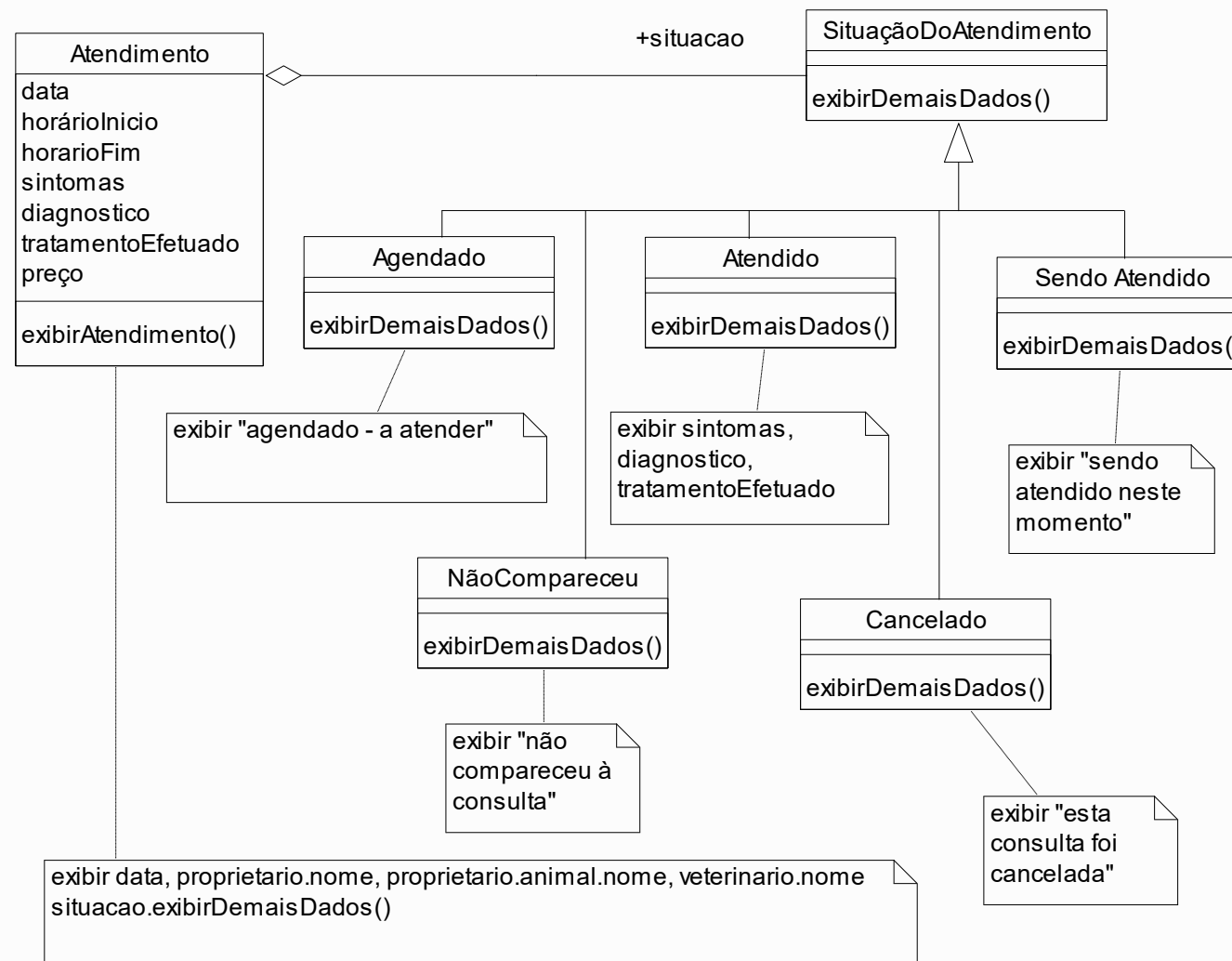
*** Imagem capturada no livro versão html publicado em CD (Gamma et al 1995)*



Padrão de Projeto Estado

- **Exemplo:**

- Exibir uma mensagem que tem uma parte fixa e outra parte que depende do estado do atendimento do paciente



Enquete 10:

Uso do padrão
Estado



Padrão de Projeto Estado

- **Questões de implementação:**
 - Quem define as transições de estado: contexto ou estado?
 - Se há um critério fixo para mudança de estado: contexto
 - Mudanças constantes: estado
 - Necessária interface para que seja possível que objetos estados modifiquem explicitamente o estado atual do contexto
 - Tabela para mapear transições
 - Criar e destruir objetos Estado
 - Criar todos os objetos a priori ou criar/destruir sob demanda?

Padrão de Projeto Estado

- Exemplo de uso do State em um jogo:
 - [State · Design Patterns Revisited · Game Programming Patterns](#)

Padrão de Projeto Estratégia (do inglês Strategy)

- **Classificação:** Padrão comportamental/Escopo de objetos
- **Intenção:** Definir uma família de algoritmos, encapsular cada um dos algoritmos e torná-los intercambiáveis. A estratégia permite que o algoritmo varie independentemente dos clientes que o usam.
- **Aplicabilidade:**
 - várias classes relacionadas diferem apenas no comportamento ou
 - são necessárias diversas versões de um algoritmo ou
 - as aplicações-cliente não precisam saber detalhes específicos de estruturas de dados de cada algoritmo

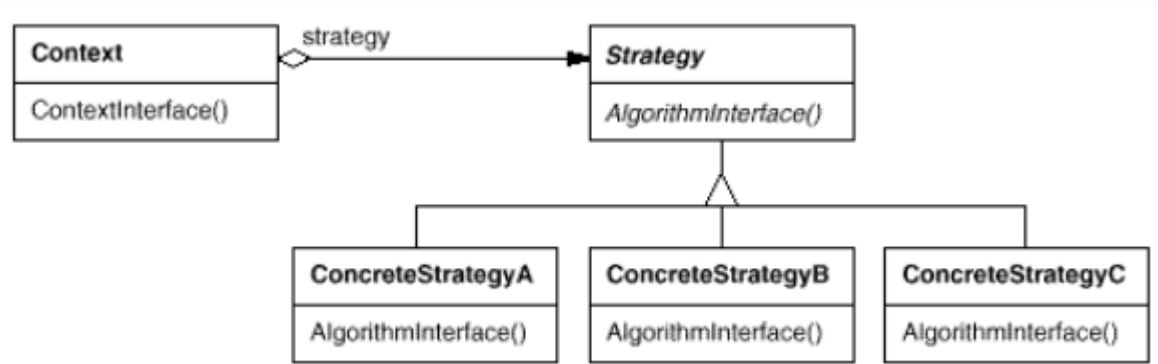


(Shvets, Alexander, 2018)

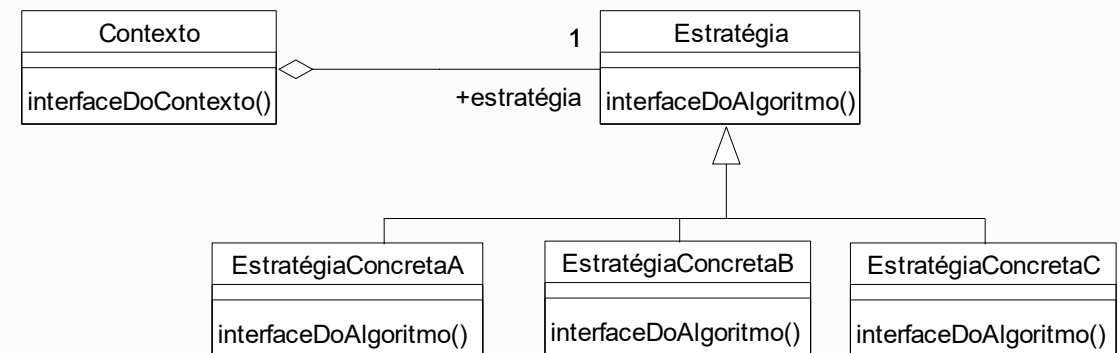
Padrão de Projeto Estratégia

- Exemplo:

- Criar uma classe abstrata para a Estratégia empregada pelo algoritmo, bem como subclasses especializando cada um dos algoritmos.
- O Contexto mantém uma referência para o objeto Estratégia e pode definir uma interface para permitir que a Estratégia acesse seus dados. A Estratégia define uma interface comum a todos os algoritmos disponíveis. O Contexto delega as solicitações recebidas das aplicações-cliente para sua estratégia.

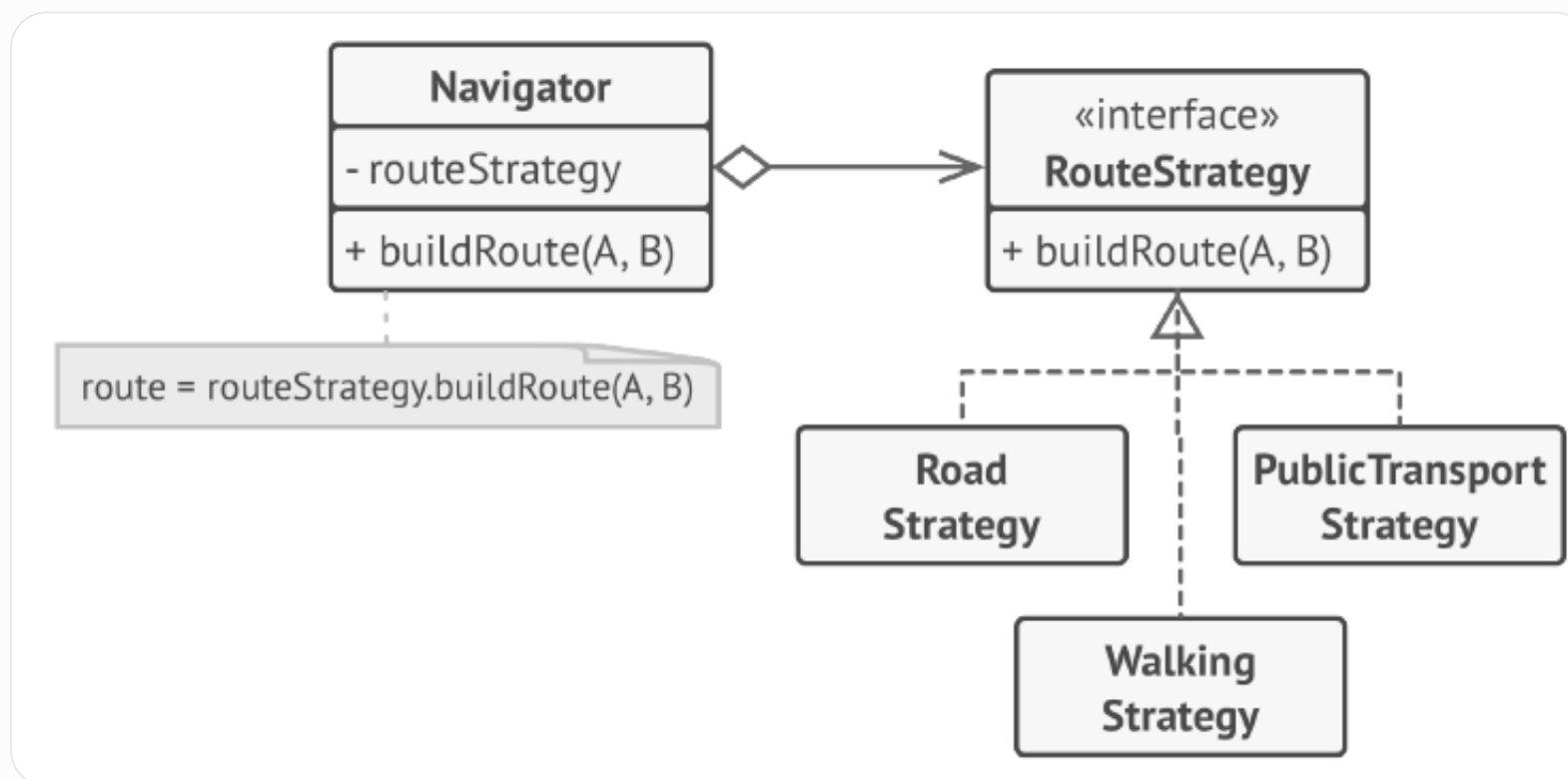


*** Imagem capturada no livro versão htm
publicado em CD (Gamma et al 1995)*



Padrão de Projeto Estratégia

- **Exemplo:** Algoritmo para cálculo de rota depende do que o usuário escolher



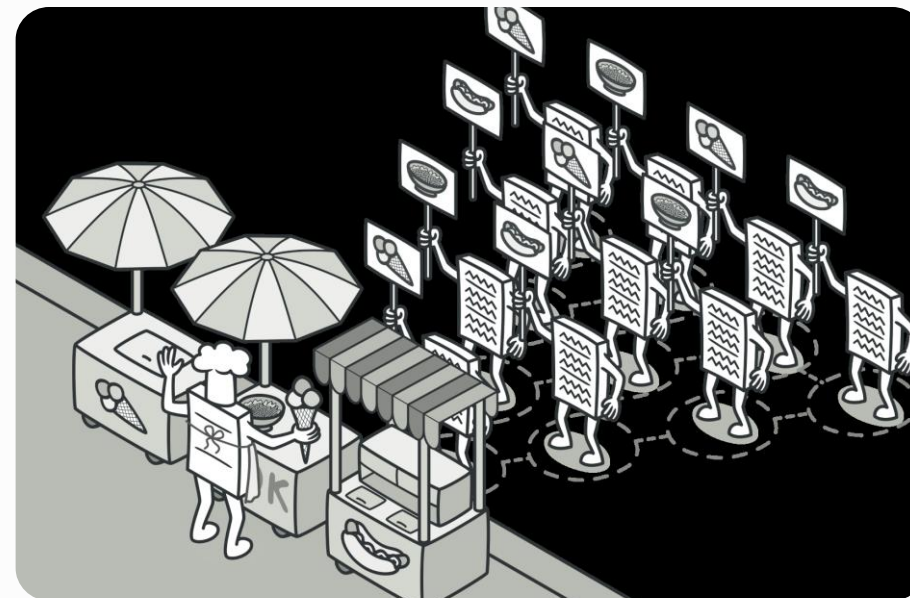
(Shvets, Alexander, 2018)

Diferença entre State e Strategy?

- State pode ser considerado uma extensão de Strategy.
- Ambos os padrões são baseados em composição: eles mudam o comportamento do contexto delegando algum trabalho para objetos auxiliares.
- Strategy torna os objetos completamente independentes e inconscientes dos outros.
- No entanto, State não restringe dependências entre estados concretos, permitindo que eles alterem o estado do contexto à vontade.

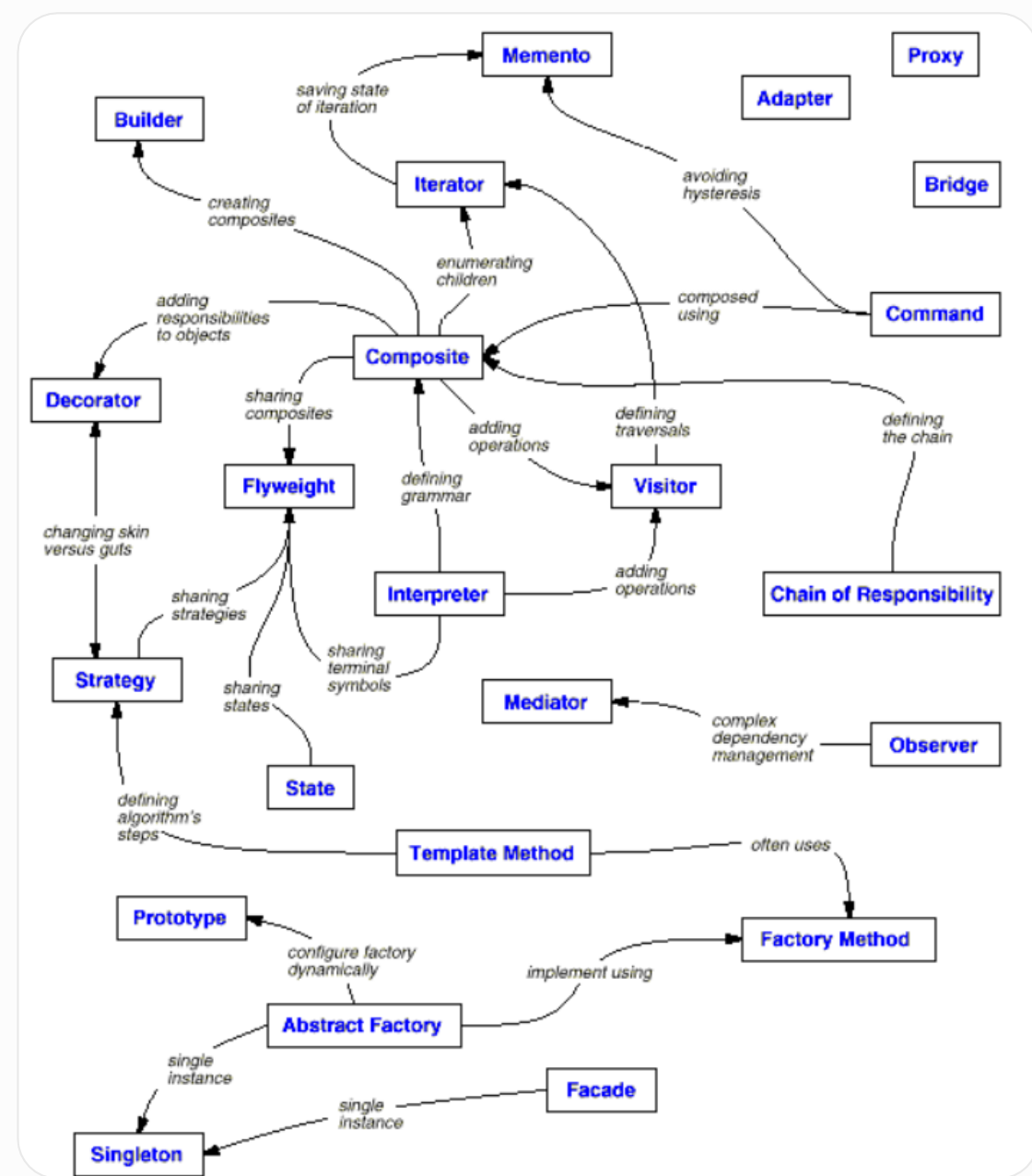
Padrão de Projeto Visitador (do inglês Visitor)

- **Classificação:** Padrão comportamental/Escopo de objetos
- **Intenção:** Permite que seja executada uma operação ao longo de um conjunto com objetos de diferentes classes, mas tendo um objeto “visitador” implementando variantes de uma mesma operação, que correspondem a todas as classes alvo.
 - Use esse padrão também quando um comportamento faz sentido apenas em algumas classes da hierarquia, mas não em outras



(Shvets, Alexander, 2018)

Relacionamentos entre os Padrões de Projeto



Conclusões

Vantagens:

- Soluções utilizadas em diversos projetos e que se provaram úteis
- Soluções com vantagens e desvantagens conhecidas
- Oferecem vocabulário comum entre desenvolvedores
- Aderem às melhores práticas de desenvolvimento OO
- Sugerem sistemas extensivos, flexíveis e robustos

Desvantagens:

- Criação de diversas classes e interfaces auxiliares, gerando mais complexidade
- Podem gerar overhead de memória e processamento
- Podem ser piores do que soluções simples quando utilizados em contextos errados

Discussão: qual é o impacto da Inteligência Artificial no desenvolvimento de software usando padrões de projeto?

- Alguns impactos visíveis:
 - Novos padrões de projeto específicos de IA: multi-agentes, prompt chaining;
 - Adaptação dos padrões de projeto tradicionais: Por exemplo:
 - 1) modelos pesados de IA podem requerer adaptações em padrões como Factory e Builder;
 - 2) a necessidade de agilidade dos agentes de IA em responder dinamicamente à mudança de condições pode requerer mudanças em padrões como Observer e Strategy.
- Impacto de padrões na sustentabilidade (mais consumo de energia)

Discussão: qual é o impacto da Inteligência Artificial no desenvolvimento de software usando padrões de projeto?

- Mudanças no ciclo de desenvolvimento
 - Auxílio na adoção de padrões de projeto: ferramentas de IA podem incorporar automaticamente esses padrões no código, melhorando as escolhas arquiteturais;
 - novas possibilidades para automatizar o processo de reconhecimento, sugestão e adaptação de padrões de projeto de software de forma eficaz, resultando na melhoria geral do desenvolvimento;
 - Problemas de manutenção: código gerado por IA pode ser mais difícil de entender/manter
 - sérios problemas podem ocorrer se os padrões não forem compreendidos pelo desenvolvedor responsável pela manutenção (aumento da dívida técnica).

Discussão: qual é o impacto da Inteligência Artificial no desenvolvimento de software usando padrões de projeto?

- Papel do engenheiro de software
 - Deixa de ser “codificador” para ser arquiteto e orquestrador (importância de uma boa especificação do software);
 - O julgamento humano é mais crítico para garantir que o código gerado pela IA seja aderente aos princípios de projeto fundamentais, incluindo padrões de projeto;
 - É fundamental que o engenheiro de software garanta que o código permaneça manutenível (reduzindo a dívida técnica).

Discussão: qual é o impacto da Inteligência Artificial no desenvolvimento de software usando padrões de projeto?

- Concluindo: a IA pode ajudar a alavancar o uso de padrões, por serem baseados em usos conhecidos para solucionar problemas reais;
- Desde o surgimento dos padrões de software, foi dito que, no futuro, ninguém precisaria se lembrar deles, pois teriam sido incorporados naturalmente ao desenvolvimento.
- A IA pode ajudar nisso, mas o papel do engenheiro de software em julgar se o padrão foi bem aplicado continua importante (pelo menos por enquanto).

Referências

- Biggerstaff, Ted J. ; Perlis, Alan J. **Software Reusability: Concepts and Models**. Acm Press Frontier Series, 1989
- Alexander et al. **A Pattern Language**. New York: Oxford University Press, 1977.
- Gamma, E.; Helm, R.; Johnson, R.; Vlissides, J. **Design Patterns - Elements of Reusable Object-Oriented Software**. Addison-Wesley, 1995
- Meszaros, G.; Doble, J. **A pattern language for pattern writing**, cap. 29 in J. Coplien; D. Schmidt. Pattern Languages of Program Design, Reading-MA, Addison-Wesley, p. 529–574, 1998.
- Buschmann F. et. al. **Pattern-oriented software architecture - a system of patterns**. Wiley, 1996.
- Coplien, J. **Advanced C++ programming styles and idioms**. Reading-MA: Addison-Wesley, 1992.
- Shvets, Alexander. **Dive into design patterns**, Refactoring.Guru, 2018, disponível para download em: <https://www.goodreads.com/book/show/43125355-dive-into-design-patterns>
- Braude, Eric. **Projeto de Software: Da programação à arquitetura: Uma abordagem baseada em Java**, Bookman, 2005.

Obrigada!

Profa. Dra. Rosana T. Vaccare Braga | www.linkedin.com/in/rosana-braga-13778912

MBA **USP**
ESALQ